A Practical Oracle-Free Preimage Attack on the 19-Round SHA-256 Compression Function via Schedule-Differential Table Lookup

Kameldip Singh Basra
Independent Researcher
kameldipbasra@gmail.com
ORCID: 0009-0001-0977-4614

June 2026

Abstract

We present two related contributions to the cryptanalysis of reduced-round SHA-256.

(1) Preimage attack. We give a practical oracle-free preimage attack on 19-round SHA-256. Given only a 32-byte target hash, the attack combines a backward-chain inversion with a precomputed 16 GB $\sigma_0(u) - u$ *differential table* and a two-stage fixed-point iteration, performing $O(2^{32})$ work per context attempt. Deduplicated measurements show that both halves of the W_9 filter carry substantial structural bias, well above the uniform 2^{-16} rate. No complete 32-bit residual match was observed in the measurement campaign, however, so the full-match rate is not yet directly estimated and we do not state one as a measured complexity claim; the marginal rates and their uncertainties are reported in §5.6. Three end-to-end attacks found verified preimages after 3, 20 and 34 contexts, which we present as demonstrations of feasibility rather than as a runtime estimator. All three were verified on a single NVIDIA H100 GPU in under 15 minutes each (cost <$1.50). For context on the practical frontier: the most recent SAT-based analysis of round-reduced SHA-256 [7] finds full-target preimages at 16, 17 and 18 rounds but reports the 19-round problem *unsolved* under all four CNF encodings it tries, and—after a 24-hour cube-and-conquer run on 16 cores—concludes that a 19-round preimage is "too difficult a task even if one carefully selects the SAT encoding and uses parallel computing," reaching 19 rounds only in a weakened form with at most 31 of the 256 output bits constrained. The present work produces three full-target 19-round preimages, each in minutes on a single GPU. The distinction is one of method rather than resources: dedicated algebraic analysis reaches a point that general-purpose search does not.

(2) Nine-Step Cancellation Lemma and partial collision. We prove a new algebraic identity: for *any* message block $W_0, \ldots, W_{15}$ and any nonzero δ, the choice $\Delta W_0 = +\delta$, $\Delta W_9 = -\delta$ forces $\Delta W_{16} = \cdots = \Delta W_{23} = 0$ exactly—an 8-round "dead zone" in the message schedule with no probabilistic conditions. Leveraging this structure, a custom CUDA kernel sweeping all 2^{32} values of a free message word at 775 million pairs per second (5.5 seconds on an RTX 4060) locates message pairs (M, M') where one 32-bit output word of the SHA-256 compression function is identical:

$$\text{SHA-256\_compress}(\text{IV}, M)[5] = \text{SHA-256\_compress}(\text{IV}, M')[5] = \texttt{0x3adcd2d0}.$$

The total differential Hamming weight is 95/256 bits (random expectation ≈ 128), and 8 distinct verified pairs are enumerated. Extending to multi-block coordinate descent reduces this further: a two-block chain achieves 75/256 (Nine-Step) or **72/256** with the partial-dead $(3, 12)$ differential. Both a three-block Nine-Step and a three-block $(13, 4)$ chain independently reach **72/256**, a 71.9% reduction from the all-different baseline. A new Schedule Density Lemma characterises the post-dead-zone schedule structure for all $|p-q|$=9 differentials and proves that partial-dead-zone pairs achieve lower floors because their interleaved-zero

positions enter the compression function earlier than the Nine-Step's post-dead-zone active rounds.

(3) The 20-round barrier for the present parameterisation. We characterise the R=20 barrier quantitatively rather than merely reporting it. The R=19 attack is shown not to schedule its constraints acyclically: it breaks a cycle by freezing W_9 provisionally, iterating, and paying one self-consistency check. Promoting a_{11} to an unknown absorbed by the fourth constraint gives R=20 the same one-word surplus, and the true R=20 solution is verified to be a fixed point of all four absorber maps—so the architecture transplants in form. It does not transplant in efficacy: measured with a single harness, the R=19 two-map iteration converges at 5.34×10^{-5} per a_0 (1,790 of 3.36×10^7) while the R=20 four-map iteration yields 0 of 2.00×10^9 ($< 1.50 \times 10^{-9}$ at 95%), below the 2^{-29} break-even. The cause is dimensional: the iterated state grows from 32 to 64 bits. We further prove that $\mathrm{rank}(\sigma_0 \oplus I) = 31$, giving a unique mask $\lambda = \texttt{0xa8a42fe4}$ with $\lambda \cdot (\sigma_0(W) \oplus W) = 0$ for every W (verified exhaustively over all 2^{32} inputs), and show that modular subtraction reduces this exact identity to a bias of only $2^{-15.8}$. Together with table coverage matching $1 - e^{-1}$ to four significant figures, this indicates that $W \mapsto \sigma_0(W) - W$ is indistinguishable from a random map under the structural tests applied, and identifies the interaction between $\mathbb{F}_2$-linear σ_0 and modular arithmetic as the mechanism by which R=20 resists.

Full SHA-256 (64 rounds) is not threatened by any of these results.

Keywords: SHA-256, preimage attack, oracle-free cryptanalysis, reduced-round, schedule-differential, Nine-Step Cancellation Lemma, Schedule Density Lemma, partial collision, birthday sweep, CUDA.

1 Introduction

SHA-256 [1] is a 64-round Merkle–Damgård compression function that underpins a broad range of security protocols. Its round-reduced variants serve as a primary benchmark for evaluating its structural security margin.

Attack model. The preimage results in this paper concern the *one-block reduced-round compression function* initialised with the standard SHA-256 IV:

$$H_R(W_0, \ldots, W_{15}) = \mathrm{IV} + \mathrm{compress}_R(\mathrm{IV}, W_0, \ldots, W_{15}).$$

The words $W_0, \ldots, W_{15}$ are arbitrary 32-bit words with no padding constraint. We make *no claim* about standard length-padded SHA-256. The *oracle-free preimage* model requires the adversary to be given only the 32-byte target hash h; no knowledge of any original preimage is assumed or used. The collision-direction experiments in Sections 8 and following use the full 64-round SHA-256 compression function, again on arbitrary one-block inputs and again without a padding constraint.

Prior work. Theoretical preimage attacks on reduced SHA-256 include the meet-in-the-middle attack of Aoki and Sasaki [2], which achieves $2^{52.3}$ for a pseudo-preimage and $2^{104.5}$ for a preimage at 24 rounds, but has never been executed concretely. Collision attacks have been extended via differential methods [4] and recently pushed to 31 steps [9], but collision attacks do not yield preimages. SAT-solver approaches [3] are effective for collision finding but do not scale to 19-round preimage search. The most relevant prior *practical* results are the SAT-based line of Pikhtovnikov and Kiryanova [5] and Davydov et al. [7], which found concrete 18-round preimages of the compression function. The latter reports 19 rounds as *unsolved* across four CNF encodings, and reaches 19 rounds only in a weakened form in which at most 31 of the 256 output bits are constrained, concluding that the full 19-round preimage problem is "too difficult a task even if one carefully selects the SAT encoding and uses parallel computing." Concurrently, Zaikin [6] reports that a parameterized Cube-and-Conquer solver, tuned on a

family of intermediate inverse problems, inverted the 29-step MD5, 24-round SHA-1 and 19-round SHA-256 compression functions. That work reaches the same round count for SHA-256 by an entirely different route. We have not been able to obtain its full text, so we do not claim priority at 19 rounds; a precise comparison of the two attack models—in particular the form of the inverted target and the compute expended—is deferred until the published version can be examined. The contribution claimed here is the algebraic method and its cost profile, not primacy.

This work. We make three contributions. The first two exploit algebraic structure in the SHA-256 message schedule; the third characterises why the first stops where it does.

Contribution 1 (Preimage, Sections 2–6): A practical oracle-free preimage attack on $R = 19$ rounds, with three concrete verified examples. Key lemmas:

1. The *W9 differential identity* (Lemma 1): a_1 cancels algebraically in the C0 schedule constraint, enabling a table-based lookup over the full 2^{32} space of a_0.
2. Analogous *unknown-cancellation identities* for a_2 and a_3 forming a fixed-point iteration (Proposition 2).
3. Three independently verified concrete 19-round preimages, found on an NVIDIA H100 in under 15 minutes each.

Contribution 2 (Collision structure, Section 8): The *Nine-Step Cancellation Lemma*: for any δ and any message block, $\Delta W_0 = +\delta$, $\Delta W_9 = -\delta$ implies $\Delta W_{16..23} = 0$ algebraically. Combined with a GPU birthday sweep, this yields a verified one-word partial collision in the SHA-256 compression function—8 explicit message pairs with a 32-bit output word identical between M and M'. Multi-block coordinate descent extends this to two- and three-block near-collisions (Hamming weight 72/256, a 71.9% reduction). The *Schedule Density Lemma* proves that for any $|p-q|=9$ differential with $\min(p, q) < 9$, three interleaved schedule positions $\Delta W_{r_0+1} = \Delta W_{r_0+3} = \Delta W_{r_0+5} = 0$ exactly, with period-7 resonance $\Delta W_{r_0+7} = \Delta W_{r_0}$; a corollary shows the full-dead-zone $(0, 9)$ pair carries nonzero differences at *all* post-dead-zone positions—the algebraic reason partial-dead-zone differentials outperform Nine-Step in the multi-block setting.

2 SHA-256 Preliminaries

Notation. All arithmetic is modulo 2^{32} unless stated otherwise. We use $\oplus$, $\&$, $\overline{x}$ for bitwise XOR, AND, NOT. Rotations and shifts of a 32-bit word x:

$$\mathrm{ROTR}^n(x) = (x \gg n) \mid (x \ll (32-n)), \qquad \mathrm{SHR}^n(x) = x \gg n.$$

Round functions.

$$\begin{aligned}
\Sigma_0(a) &= \mathrm{ROTR}^2(a) \oplus \mathrm{ROTR}^{13}(a) \oplus \mathrm{ROTR}^{22}(a),\\
\Sigma_1(e) &= \mathrm{ROTR}^6(e) \oplus \mathrm{ROTR}^{11}(e) \oplus \mathrm{ROTR}^{25}(e),\\
\sigma_0(x) &= \mathrm{ROTR}^7(x) \oplus \mathrm{ROTR}^{18}(x) \oplus \mathrm{SHR}^3(x),\\
\sigma_1(x) &= \mathrm{ROTR}^{17}(x) \oplus \mathrm{ROTR}^{19}(x) \oplus \mathrm{SHR}^{10}(x),\\
\mathrm{Ch}(e, f, g) &= (e \,\&\, f) \oplus (\overline{e} \,\&\, g),\\
\mathrm{Maj}(a, b, c) &= (a \,\&\, b) \oplus (a \,\&\, c) \oplus (b \,\&\, c).
\end{aligned}$$

State labels. We write the 8-word compression state after round r as $(a_r, a_{r-1}, a_{r-2}, a_{r-3}, e_r, e_{r-1}, e_{r-2}, e_{r-3})$, with IV boundary values $a_{-4}, \ldots, a_{-1}$ and $e_{-4}, \ldots, e_{-1}$:

a_{-1}=IV[0], a_{-2}=IV[1], a_{-3}=IV[2], a_{-4}=IV[3], e_{-1}=IV[4], e_{-2}=IV[5], e_{-3}=IV[6], e_{-4}=IV[7].

Round equations.

$$T_1^{(r)} = \Sigma_1(e_{r-1}) + \mathrm{Ch}(e_{r-1}, e_{r-2}, e_{r-3}) + e_{r-4} + K_r + W_r, \tag{1}$$

$$T_2^{(r)} = \Sigma_0(a_{r-1}) + \mathrm{Maj}(a_{r-1}, a_{r-2}, a_{r-3}), \tag{2}$$

$$a_r = T_1^{(r)} + T_2^{(r)}, \tag{3}$$

$$e_r = a_{r-4} + T_1^{(r)}. \tag{4}$$

Note that e_{r-4} in (1) is the *h-register* word (boundary value $e_{-4} = \mathrm{IV}[7]$), while a_{r-4} in (4) is the *d-register* word (boundary value $a_{-4} = \mathrm{IV}[3]$); these are distinct IV words and distinct chain values for all $r \geq 4$.

From (3)–(4): $T_1^{(r)} = a_r - T_2^{(r)}$, and rearranging (1):

$$W_r = T_1^{(r)} - \Sigma_1(e_{r-1}) - \mathrm{Ch}(e_{r-1}, e_{r-2}, e_{r-3}) - e_{r-4} - K_r. \tag{5}$$

Message schedule. For $r \geq 16$ the message schedule expands:

$$W_r = \sigma_1(W_{r-2}) + W_{r-7} + \sigma_0(W_{r-15}) + W_{r-16}. \tag{6}$$

For $r < 16$ the word W_r is an independent input to the compression function; it is what we seek.

Hash output. After R rounds the hash is $h[i] = \mathrm{IV}[i] + s_i$ for $i = 0, \ldots, 7$, where the final state vector is $(s_0, \ldots, s_7) = (a_{R-1}, a_{R-2}, a_{R-3}, a_{R-4}, e_{R-1}, e_{R-2}, e_{R-3}, e_{R-4})$.

3 Structural Analysis

3.1 Backward Chain

From the R-round hash we recover the final state directly:

$$a_{R-1}, a_{R-2}, a_{R-3}, a_{R-4},\ e_{R-1}, e_{R-2}, e_{R-3}, e_{R-4}.$$

We then extend backward using $T_2^{(r)} = \Sigma_0(a_{r-1}) + \mathrm{Maj}(a_{r-1}, a_{r-2}, a_{r-3})$, $T_1^{(r)} = a_r - T_2^{(r)}$, $a_{r-4} = e_r - T_1^{(r)}$, iterating for $r = R-1, R-2, R-3, R-4$ to recover $a_{R-5}, \ldots, a_{R-8}$.

For $R = 19$ this yields $\{a_{11}, a_{12}, \ldots, a_{18}\}$: the state words a_{11} through a_{18} are determined by the target hash alone, with no knowledge of any preimage.

3.2 Free Parameters and Schedule Constraints

At $R = 19$ the unconstrained parameters are $a_0, \ldots, a_{10}$ (11 words). Of these, $a_4, \ldots, a_{10}$ are chosen uniformly at random per attempt (*context*), forming 7 attacker-controlled free words. The remaining $a_0, \ldots, a_3$ are the targets of the attack phases.

The schedule (6) first activates at $r = 16$, yielding three consistency constraints at $R = 19$:

$$W_{16} = \sigma_1(W_{14}) + W_9 + \sigma_0(W_1) + W_0, \tag{C0}$$

$$W_{17} = \sigma_1(W_{15}) + W_{10} + \sigma_0(W_2) + W_1, \tag{C1}$$

$$W_{18} = \sigma_1(W_{16}) + W_{11} + \sigma_0(W_3) + W_2. \tag{C2}$$

Here W_{16}, W_{17}, W_{18} are recovered oracle-free from the backward chain via (5) using the known $a_{11}, \ldots, a_{18}$ together with the context values $a_4, \ldots, a_{10}$ (which supply $e_{11}, \ldots, e_{14}$, needed to evaluate (5) for $r = 16, 17, 18$). Similarly W_{14} and W_{15} are recovered from the backward chain and context.

4 The σ_0-Differential Table

Definition 1. *The* σ_0-differential table $\mathcal{T}$ *is an array of* 2^{32} *32-bit entries:*

$$\mathcal{T}[v] = u$$

where u *is a representative satisfying* $\sigma_0(u) - u \equiv v \pmod{2^{32}}$*, or a sentinel* $\perp$ *if no such* u *exists. When multiple* u *share the same* v*, one representative is stored.*

Construction. $\mathcal{T}$ is built in a single pass: for each $u \in [0, 2^{32})$, compute $v = (\sigma_0(u) - u) \bmod 2^{32}$ and write u to $\mathcal{T}[v]$. On an NVIDIA H100 in chunks of 2^{25}, the build takes under 2 seconds and requires 16 GB of GPU VRAM.

Coverage. The function $f(u) = \sigma_0(u) - u \bmod 2^{32}$ is not injective. Empirically, 2,721,603,628 of the 2^{32} possible values v have at least one preimage, corresponding to **63.37%** coverage.

Usage. Given a target value v^*, the query $\mathcal{T}[v^*]$ returns in $O(1)$ a word u satisfying $\sigma_0(u) - u = v^*$, or $\perp$ (probability $\approx 36.6\%$).

5 The R=19 Oracle-Free Preimage Attack

5.1 Overview

The attack operates in three phases per context:

C0. Sweep $a_0 \in [0, 2^{32})$. A table lookup recovers W_1 (and hence a_1) for 63.37% of a_0 values.

C1/C2. For each C0 survivor, run a fixed-point iteration: table lookup on the W_{17} constraint recovers a_2; table lookup on the W_{18} constraint recovers a_3; repeat until convergence.

W9 filter. Verify the W_9 consistency condition ($1/2^{32}$ pass rate) to gate final forward verification.

The key that makes all three phases work is a common algebraic pattern: in each constraint, the target unknown a_i appears with *opposite signs* in two different terms, causing exact cancellation in the σ_0-differential lookup target.

5.2 Context Precomputation

Fix a target hash h. Run the backward chain to obtain $a_{11}, \ldots, a_{18}$. Choose uniformly random context $a_4, \ldots, a_{10}$. Precompute the following constants using the context values and backward-chain values:

$$T_2^{\mathrm{IV}} = \Sigma_0(a_{-1}) + \mathrm{Maj}(a_{-1}, a_{-2}, a_{-3}), \tag{7}$$

$$C_0 = -T_2^{\mathrm{IV}} - e_{-4} - \Sigma_1(e_{-1}) - \mathrm{Ch}(e_{-1}, e_{-2}, e_{-3}) - K_0, \tag{8}$$

$$C_{e_0} = a_{-4} - T_2^{\mathrm{IV}}. \tag{9}$$

From these: $W_0 = a_0 + C_0$ and $e_0 = a_0 + C_{e_0}$, both linear in a_0.

For the context rounds $r = 7, \ldots, 11$, compute $T_2^{(r)}, T_1^{(r)}, e_r$ from (1)–(4) using the context $a_4, \ldots, a_{11}$ (the backward chain supplies a_{11}). Define:

$$W_9^{\mathrm{base}} = T_1^{(9)} - \Sigma_1(e_8) - K_9, \tag{10}$$

$$W_{11}^{\mathrm{base}} = T_1^{(11)} - T_1^{(7)} - \Sigma_1(e_{10}) - \mathrm{Ch}(e_{10}, e_9, e_8) - K_{11}, \tag{11}$$

$$CONST_{10} = T_1^{(10)} - \Sigma_1(e_9) - K_{10}. \tag{12}$$

Also recover $W_{14}, W_{15}, W_{16}^{\text{bc}}, W_{17}^{\text{bc}}, W_{18}^{\text{bc}}$ from (5) at rounds $14, 15, 16, 17, 18$ respectively (all oracle-free from backward chain + context).

Finally, initialise $\hat{W}_9$ with $a_2 = a_3 = 0$:

$$\hat{W}_9 = W_9^{\text{base}} - T_1^{(5)}\big|_{a_3=0} - \text{Ch}\Big(e_8,\, T_1^{(7)} + 0,\, T_1^{(6)}\big|_{a_2=a_3=0}\Big), \tag{13}$$

where $T_1^{(5)}\big|_{a_3=0}$ denotes $T_1^{(5)}$ evaluated with the sweep variable a_3 set to zero (and $a_2 = 0$).

5.3 Phase C0: The W9 Differential Identity

The core algebraic insight that makes the C0 phase work is the following.

Lemma 1 (W9 Differential)**.** *Let W_9 be the true message word derived from the preimage, and let $\hat{W}_9$ be the approximation computed by* (13) *with $a_2 = a_3 = 0$. Then:*

$$\hat{W}_9 - W_9 = a_1 \pmod{2^{32}}.$$

Proof. Apply (5) at $r = 9$:

$$W_9 = \underbrace{T_1^{(9)} - \Sigma_1(e_8) - K_9}_{W_9^{\text{base}}} - e_5 - \text{Ch}(e_8, e_7, e_6).$$

$T_1^{(9)}$, e_8 depend only on context $(a_4, \ldots, a_9)$, so W_9^{base} is fully determined by context.

Shift-register decomposition. The identity $e_r = a_{r-4} + T_1^{(r)}$ (from (3)–(4)) applied at $r = 5, 6, 7$ yields:

$$e_5 = a_1 + T_1^{(5)}, \tag{14}$$

$$e_6 = a_2 + T_1^{(6)}, \tag{15}$$

$$e_7 = a_3 + T_1^{(7)}, \tag{16}$$

where $T_1^{(7)} = a_7 - T_2^{(7)}$ is context-only ($T_2^{(7)} = \Sigma_0(a_6) + \text{Maj}(a_6, a_5, a_4)$), and $T_1^{(5)}, T_1^{(6)}$ carry the a_2, a_3 dependence through $T_2^{(5)}, T_2^{(6)}$.

Substituting (14) into W_9:

$$W_9 = W_9^{\text{base}} - a_1 - T_1^{(5)} - \text{Ch}(e_8, e_7, e_6).$$

Approximation. $\hat{W}_9$ (13) evaluates this expression with $a_2 = a_3 = 0$ *and* with the a_1 term omitted from e_5:

$$\hat{W}_9 = W_9^{\text{base}} - T_1^{(5)}\big|_{a_2=a_3=0} - \text{Ch}\big(e_8,\, T_1^{(7)},\, T_1^{(6)}\big|_{a_2=a_3=0}\big).$$

Hence

$$\hat{W}_9 - W_9 = a_1 + \big[T_1^{(5)} - T_1^{(5)}\big|_{a_2=a_3=0}\big] + \big[\text{Ch}(e_8, e_7, e_6) - \text{Ch}\big(e_8, T_1^{(7)}, T_1^{(6)}\big|_{a_2=a_3=0}\big)\big].$$

The two bracket terms capture how a_2, a_3 perturb $T_1^{(5)}$ (through $\text{Maj}(a_4, a_3, a_2)$) and the Ch argument (through e_6, e_7). These perturbations are opposite in sign and cancel exactly: the a_3 contribution enters e_7 additively while $T_1^{(6)}(a_3)$ carries the same shift in the opposite direction of the Ch output; the a_2 contribution similarly cancels via e_6. The combined cancellation leaves $\hat{W}_9 - W_9 = a_1$.

Numerical confirmation. For all three independently verified preimages (Table 2) the identity was checked:

$$\text{P1: } \hat{W}_9 - W_9 = \texttt{0x72c05667} - \texttt{0x6d1e5a20} = \texttt{0x05a1fc47} = a_1.$$

The identity also holds for $> 10^4$ random contexts in the reproducibility package. □

C0 lookup. For each $a_0 \in [0, 2^{32})$, compute:

$$e_0 = a_0 + C_{e_0}, \tag{17}$$
$$W_0 = a_0 + C_0, \tag{18}$$
$$g(a_0) = -\Sigma_0(a_0) - \mathrm{Maj}(a_0, a_{-1}, a_{-2}) - e_{-3} - \Sigma_1(e_0) - \mathrm{Ch}(e_0, e_{-1}, e_{-2}) - K_1, \tag{19}$$
$$F_0 = W_{16}^{\mathrm{bc}} - \sigma_1(W_{14}) - g(a_0) - \hat{W}_9 - W_0. \tag{20}$$

Note that $g(a_0) = W_1 - a_1$: it is the portion of W_1 that depends only on a_0 (derived from the round-1 inverse equation, with a_1 factored out). Then:

Proposition 1 (C0 cancellation)**.** $F_0 = \sigma_0(W_1) - W_1 \pmod{2^{32}}$.

Proof. From the C0 schedule constraint and Lemma 1, $W_9 = \hat{W}_9 - a_1$, so

$$\sigma_0(W_1) = W_{16}^{\mathrm{bc}} - \sigma_1(W_{14}) - W_9 - W_0 = W_{16}^{\mathrm{bc}} - \sigma_1(W_{14}) - \hat{W}_9 + a_1 - W_0.$$

Since $W_1 = a_1 + g(a_0)$:

$$\sigma_0(W_1) - W_1 = W_{16}^{\mathrm{bc}} - \sigma_1(W_{14}) - \hat{W}_9 + a_1 - W_0 - a_1 - g(a_0) = F_0. \qquad \square$$

The a_1 terms cancel exactly. A single table lookup $\mathcal{T}[F_0]$ returns W_1, and then $a_1 = W_1 - g(a_0)$.

5.4 Phases C1/C2: Fixed-Point Iteration

For each C0 survivor (a_0, a_1, W_1), derive:

$$e_1 = e_{-3} + a_1 - \Sigma_0(a_0) - \mathrm{Maj}(a_0, a_{-1}, a_{-2}), \tag{21}$$
$$T_2^{(2)} = \Sigma_0(a_1) + \mathrm{Maj}(a_1, a_0, a_{-1}), \tag{22}$$
$$F_{12} = -T_2^{(2)} - e_{-2} - \Sigma_1(e_1) - \mathrm{Ch}(e_1, e_0, e_{-1}) - K_2. \tag{23}$$

Note that $F_{12} = W_2 - a_2$: the portion of W_2 depending only on (a_0, a_1).

Proposition 2 (C1 and C2 cancellations)**.** *Define the a_3-dependent quantity:*

$$D(a_3) = \big(a_6 - \Sigma_0(a_5) - \mathrm{Maj}(a_5, a_4, a_3)\big) + \mathrm{Ch}\big(e_9,\, e_8,\, a_3 + T_1^{(7)}\big).$$

Let $W_{16}^{\mathrm{sched}} = \sigma_1(W_{14}) + \hat{W}_9 + \sigma_0(W_1) + W_0$ and $W_{16}^{*} = W_{16}^{\mathrm{sched}} - a_1$. Then:

$$F_{\mathrm{C1}}(a_3) = W_{17}^{\mathrm{bc}} - \sigma_1(W_{15}) - W_1 - CONST_{10} - F_{12} + D(a_3) = \sigma_0(W_2) - W_2, \quad \text{(C1-target)}$$
$$F_{\mathrm{C2}}(a_2, a_3) = W_{18}^{\mathrm{bc}} - \sigma_1(W_{16}^{*}) - W_{11}^{\mathrm{base}} - W_2 - F_{23}(a_0, a_1, a_2) = \sigma_0(W_3) - W_3, \quad \text{(C2-target)}$$

where $F_{23}(a_0, a_1, a_2)$ is the portion of W_3 depending on (a_0, a_1, a_2) but not a_3.

Proof. **C1.** From (5) at $r = 10$, using $CONST_{10} = T_1^{(10)} - \Sigma_1(e_9) - K_{10}$:

$$W_{10} = CONST_{10} - e_6 - \mathrm{Ch}(e_9, e_8, e_7).$$

By the shift-register identity $e_r = a_{r-4} + T_1^{(r)}$: $e_6 = a_2 + T_1^{(6)}(a_3)$ and $e_7 = a_3 + T_1^{(7)}$, so $\mathrm{Ch}(e_9, e_8, e_7) = \mathrm{Ch}(e_9, e_8, a_3 + T_1^{(7)})$. Grouping the a_3-dependent terms into $D(a_3)$:

$$W_{10} = CONST_{10} - a_2 - D(a_3).$$

Substituting into the C1 schedule constraint $\sigma_0(W_2) = W_{17}^{\mathrm{bc}} - \sigma_1(W_{15}) - W_{10} - W_1$:

$$\sigma_0(W_2) = W_{17}^{\mathrm{bc}} - \sigma_1(W_{15}) - (CONST_{10} - a_2 - D(a_3)) - W_1 = W_{17}^{\mathrm{bc}} - \sigma_1(W_{15}) - CONST_{10} + a_2 + D(a_3) - W_1.$$

Using $W_2 = a_2 + F_{12}$, the a_2 terms cancel:

$$\begin{aligned}\sigma_0(W_2) - W_2 &= \big(W_{17}^{\text{bc}} - \sigma_1(W_{15}) - \mathit{CONST}_{10} + a_2 + D(a_3) - W_1\big) - (a_2 + F_{12}) \\ &= W_{17}^{\text{bc}} - \sigma_1(W_{15}) - W_1 - \mathit{CONST}_{10} - F_{12} + D(a_3) = F_{\text{C1}}(a_3). \quad \square_{\text{C1}}\end{aligned}$$

C2. From (5) at $r = 11$: $W_{11} = T_1^{(11)} - \Sigma_1(e_{10}) - \text{Ch}(e_{10}, e_9, e_8) - e_7 - K_{11}$. Since $e_7 = a_3 + T_1^{(7)}$:

$$W_{11} = \underbrace{\big[T_1^{(11)} - T_1^{(7)} - \Sigma_1(e_{10}) - \text{Ch}(e_{10}, e_9, e_8) - K_{11}\big]}_{W_{11}^{\text{base}}} - a_3.$$

So $W_{11} = W_{11}^{\text{base}} - a_3$ with W_{11}^{base} determined by context alone.

By Lemma 1, $W_{16} = W_{16}^{\text{sched}} - a_1 = W_{16}^*$, so $\sigma_1(W_{16}) = \sigma_1(W_{16}^*)$. Substituting into C2: $\sigma_0(W_3) = W_{18}^{\text{bc}} - \sigma_1(W_{16}^*) - W_{11} - W_2 = W_{18}^{\text{bc}} - \sigma_1(W_{16}^*) - (W_{11}^{\text{base}} - a_3) - W_2$. Using $W_3 = a_3 + F_{23}$, the a_3 terms cancel:

$$\begin{aligned}\sigma_0(W_3) - W_3 &= W_{18}^{\text{bc}} - \sigma_1(W_{16}^*) - W_{11}^{\text{base}} + a_3 - W_2 - a_3 - F_{23} \\ &= W_{18}^{\text{bc}} - \sigma_1(W_{16}^*) - W_{11}^{\text{base}} - W_2 - F_{23} = F_{\text{C2}}(a_2, a_3). \quad \square_{\text{C2}}\end{aligned}$$

□

Iteration. The joint C1/C2 system defines a map

$$(a_2, a_3) \;\mapsto\; \big(\mathcal{T}[F_{\text{C1}}(a_3)] - F_{12},\; \mathcal{T}[F_{\text{C2}}(a_2, a_3)] - F_{23}\big).$$

We iterate from a seed $(a_2^{(0)}, a_3^{(0)})$ for up to 8 steps. A *convergent pair* is one at which both C1 and C2 are satisfied simultaneously, i.e., the map is a fixed point. Empirical convergence rate: $\approx 2.3 \times 10^{-5}$ per C0 survivor.

K-seed optimisation. Pre-computing K initial pairs (a_2, a_3) that satisfy the low-16-bit constraint $\hat{W}_9(a_2, a_3) \bmod 2^{16} = \hat{W}_9 \bmod 2^{16}$ improves the convergence rate by $\approx 3.3\times$.

5.5 W9 Bit-Filter

After each convergent pair (a_2, a_3) is found, recompute the *true* W_9 using the converged values:

$$W_9^{\text{true}} = W_9^{\text{base}} - \big(a_5 - \Sigma_0(a_4) - \text{Maj}(a_4, a_3, a_2)\big) - \text{Ch}\big(e_8,\; a_3 + T_1^{(7)},\; a_2 + T_1^{(6)}(a_3)\big). \tag{24}$$

Two sequential filters are applied:

- **Lo filter:** $(W_9^{\text{true}} \bmod 2^{16}) = (\hat{W}_9 \bmod 2^{16})$. Passes at 52–88× (E1) or 37–44× (E2) the uniform 2^{-16} rate (§5.6).
- **Hi filter:** $(W_9^{\text{true}} \gg 16) = (\hat{W}_9 \gg 16)$. Passes at 12.6–13.3× (E1) or 7–11× (E2) the uniform rate. This filter is *not* uniform; a coarse-binned test suggests otherwise but is underpowered against a spike at a single residual (§6.3).

The combined rate is not obtained by multiplying the two marginals: the iteration is seeded on lo-consistency, so independence of the two halves cannot be assumed and has not been established (§5.6). Candidates that pass both filters are assembled into a full 16-word message $W_0, \ldots, W_{15}$ and verified by forward evaluation of the 19-round compression function.

Table 1: Empirical per-context statistics (H100, batch 2^{24}, averaged over > 100 contexts for per-context quantities; 165-context hi-residual run for lo-pass count and chi-squared test).

Quantity	Value
C0 survivors (table hits)	2,721,600,000 (63.4% of 2^{32})
E1 *— deduplicated (unique fixed points); two CPU pilots*	
C1/C2 fixed points	3.6–5.1×10^4 per context
W_9 lo-pass events	28–68 per context
W_9 lo-pass rate per fixed point	7.9×10^{-4}–1.3×10^{-3}
Structural *lo* enhancement	52–88× above 2^{-16}
W_9 hi-pass rate per fixed point	$\approx 2.0 \times 10^{-4}$
Structural *hi* enhancement	12.6–13.3× above 2^{-16}
Legacy trajectory counts (each fixed point counted once per remaining iteration)	
Repeat-count inflation factor	5.5–5.9×
μ (original model: measured lo count × uniform conditional hi)	$\approx 5 \times 10^{-4}$
μ (directly measured)	not yet estimated ($N_{LH} = 0$)
μ (independence-derived projection)	6×10^{-3}–1.3×10^{-2}
Observed contexts (P3, P2, P1)	3, 20, 34 (mean 19)
Wall time per context (H100)	≈ 25 s

5.6 Complexity Analysis

W9 structural enhancement. Neither half of the W_9 filter is uniform. Both enhancements arise because the C1/C2 fixed-point iteration, started from W_9-lo-consistent seed pairs, preferentially converges to fixed points that also satisfy the W_9 constraint—an effect that acts on the full 32-bit residual, not only on the low half. Experiment **E2**, an independent CPU reimplementation measuring over 2.2×10^9 swept a_0 values per configuration, gives:

- the *lo* filter passes at 37–44× the uniform 2^{-16} rate, confirming the enhancement mechanism;
- the *hi* filter passes at 7–11× the uniform rate—it is *not* uniform, contrary to what a coarse-binned test suggests (§6.3).

E1 and E2 are separate campaigns and their rates are not directly comparable: E1 deduplicates fixed points and reports per-fixed-point rates from two CPU pilots, whereas E2 reports per-swept-a_0 rates from a distinct implementation on different contexts. Both find the same qualitative effect — a large *lo* enhancement and a smaller but nonzero *hi* enhancement — and the residual factor of roughly 1.5 between them is not currently explained; it is within the spread expected from context-to-context variation (§5.7), which is itself large. **E1 is the headline dataset**, since it is deduplicated and feeds the μ projection. Neither campaign is yet a preregistered success-rate measurement; that experiment remains outstanding.

These rates are not artefacts of the lookup table's representative policy. Repeating the measurement on byte-identical contexts with a table built to keep the *smallest* rather than the *largest* preimage of each target gives lo enhancement 30.0× against 28.5×, and hi 11.1× against 6.9×, agreeing within Poisson error on the 8 and 13 hi events observed. Direct tests of lo/hi independence at reduced filter widths give $D = P(\text{both})/[P(\text{lo})P(\text{hi})] = 1.00 \pm 0.03$ at 4 bits and 1.06 ± 0.10 at 6 bits, consistent with independence at the widths where joint events can be counted.

Multiplying the measured marginals by the deduplicated fixed-point count gives a *projection* of $\mu \approx 6 \times 10^{-3}$ per context. Once fixed points are deduplicated the lo-pass count is 28–35 per context, close to the figure obtained from the original GPU diagnostic, so the lo analysis stands;

the correction is almost entirely the hi filter, worth about 13× rather than the two orders of magnitude a naive comparison of non-deduplicated counts suggests. We label this a projection rather than a measurement for two reasons. First, it assumes the two halves are independent; since the iteration is seeded on lo-consistency, that is exactly the assumption one should not make for free. Half-width diagnostics detect no material dependence up to 12 bits, but a sparse spike at the exact 16-bit value would remain invisible to them—the same limitation that makes the binned χ^2 test uninformative (§6.3). Second, no complete 32-bit match was observed, so the direct estimator $\hat{\mu} = N_{LH}/C$ has no events to work with; over C context-equivalents with $N_{LH} = 0$ the one-sided 95% Poisson bound is $\mu < 2.996/C$. The available projections are also unstable. After adopting global fixed-point deduplication, the independence-derived projection was 1.31×10^{-2} in a 0.20 context-equivalent pilot and 5.80×10^{-3} in a 1.37 context-equivalent pilot. That variation shows the marginal-event sample is too small for a stable projection; it does not by itself establish a directional bias. An earlier figure of 4.88×10^{-2} used repeat-counted fixed points and is not comparable with either.

Two earlier sources of error are worth recording, since both inflate μ if left uncorrected. A converged lane remains at its fixed point for every subsequent iteration, so counting convergences per iteration overcounts each fixed point—measured at 5.89× for eight iterations. And convergences tallied per seed chain rather than per context understate the per-context count by the number of chains. All figures above are globally deduplicated on the fixed-point identity (a_0, a_1, a_2, a_3) within a context.

What the three finds do and do not establish. The three preimages were found after 3, 20 and 34 contexts: three successes over 57 attempted contexts, $\hat{p} = 0.053$ with an exact 95% interval of [0.011, 0.146]. The 6×10^{-3} projection lies *below* that interval. Under $p = 0.006$, observing three or more successes in 57 contexts has probability 0.005, so either the historical runs were unusually favourable or the projection is low. Two very small samples are in tension here, and neither settles the rate. It is not compatible with the uniform-filter model: at $p \approx 5 \times 10^{-4}$ the same observation has probability 3×10^{-7}, so the finds do falsify the uniform model even though they cannot pin the true rate. We therefore present them as demonstrations of feasibility, not as a runtime estimator. Three observations cannot support an expected-value claim, and the per-context success probability $\Pr(N_{LH} \geq 1)$—which governs time to first preimage—is not in general μ itself if successful fixed points cluster in favourable contexts. A preregistered multi-target campaign reporting both quantities is in progress. The combined search time for the three was 1,464 s on the H100 (871.6, 511.1 and 81.7 s).

5.7 Context Screening

The clustering just hypothesised is real and is large enough to exploit. The attack samples a context and then sweeps a_0 over 2^{32}, giving every context the same budget; that is optimal only if contexts are interchangeable. Over 280 sampled contexts the deduplicated C1/C2 fixed-point count per context had mean 63.4 and variance 97,082, a variance-to-mean ratio of ≈1,530 against the 1.0 that a Poisson model would give, with median 12 and maximum 4,924. The top decile of contexts held 68.6% of the total yield.

Overdispersion alone is not usable, since it could be per-sample noise. The operative question is whether a cheap score measured on one a_0 sample predicts yield on an independent one. Scoring each of 392 contexts twice on independent 2^{20} samples gave a Spearman rank correlation of 0.910 between the two scores, so productivity is a stable property of the context and is recoverable from a short pass. Since screening costs 2^{20} against a full sweep of 2^{32}, it is 1/4096 of a context's cost and is nearly free: selecting the top 1% on the screening sample gave a 12.99× out-of-sample yield lift for a 2.44% overhead, a net factor of 12.68×. Nothing in this construction depends on the round count, so it applies unchanged at $R = 20$.

Does the lift survive conversion? The score counts C1/C2 fixed points, so the lift above is in the first place a fixed-point lift. The operative question is whether contexts selected for fixed-point productivity also yield more of the rarer downstream events, or whether they merely produce more fixed points of lower individual quality. A replication (`screening_validation.py`, 392 contexts scored twice on independent 2^{20} samples, seed 99001) reproduced the correlation at $\rho = 0.929$ and produced 22 lo-pass events, enough to test the conversion directly. Selecting on the sample-1 fixed-point score and counting sample-2 lo-passes—so that selection and measurement never share data—gives:

Keep	lo events	expected	lift
top 25%	7 of 10	2.50	2.80×
top 10%	5 of 10	1.00	5.03×
top 5%	3 of 10	0.50	5.88×

Under the null that screening carries no information about lo-passes, each event falls in the top decile with probability 0.1; observing 5 of 10 has $p = 1.6 \times 10^{-3}$ (and the reverse direction, selecting on sample 2 and measuring sample 1, gives 9 of 12, $p = 1.7 \times 10^{-7}$; the two directions share data and are not independent replicates). The lift therefore does carry through, and does so at roughly the fixed-point lift (5.0× against 6.8× at the top decile), i.e. lo-passes scale about proportionally with fixed points rather than being diluted.

We nonetheless state the remaining scope limit precisely. What is now supported is that screening raises *lo-pass* yield per attacked context. What is still *not* established is conversion through the *hi* filter to a full 32-bit W_9 match, and thence to a preimage—the rarest and decisive step, on which these samples produced no events. This is therefore **not yet a measured speedup of the attack**, and the preregistered paired experiment that would settle it—screened and random-control arms given identical 2^{32} exposure, with N_{LH}/C as the estimator—is outstanding. Finally, the deep tail is unstable: the top-1% row rests on 4 contexts of 392 and moved from 12.99× to 31.93× between runs, so we quote the top-decile figure (4.98× and 6.79× in the two runs) as the defensible one.

6 Experimental Results

6.1 Hardware and Software

All experiments were run on NVIDIA H100 SXM5 80 GB HBM3 via RunPod cloud compute. The attack code uses PyTorch 2.x on CUDA, with the C0 sweep vectorised in batches of 2^{24} elements. The 16 GB σ_0-differential table is held in GPU memory throughout the sweep; table lookups are implemented as indexed tensor reads.

6.2 Verified Preimages

Each row of Table 2 was independently verified by the following procedure:

1. Parse the 16 words $W_0, \ldots, W_{15}$ as a 512-bit block.
2. Run the standard SHA-256 message schedule for 19 rounds, initialised with the standard IV.
3. Add the final state to the IV word-by-word.
4. Confirm the resulting 32-byte value equals the target hash.

All three preimages pass this check. The verifier is included in the accompanying reproducibility package.

Table 2: Verified oracle-free R=19 preimages. All three were found with no knowledge of any original message; only the target hash was provided to the solver. All values are hexadecimal. †Context 3; a_0 sweep reached 22% before the hit.

	Hash / words	**Time / note**
P1	`1e65261c 54255188 604f5375 09183973`	871.6 s
	`3de63e96 6b5e4715 658226bf 03588447`	ctx 34
	`22f091af ec52d67b 74c33819 a280dc6a`	W_0–W_7
	`b001ff1a 1f2356a5 3eccf108 bd9a2333`	
	`abe611d1 6d1e5a20 8041df25 e43d31af`	W_8–W_{15}
	`aa895a2e 69106ad2 7479fa3a 2a9abb91`	
P2	`fb52f81b aed24f87 28faf5bb ce82c67d`	511.1 s
	`51076117 2fb9876d 9e3a72dd a351b7ca`	ctx 20
	`37e6702f bc20efea 2dd42a3e 501dfbe9`	W_0–W_7
	`3cacc578 ea2de1c1 11c0f066 0f22be47`	
	`2a447d2d 13f0080f 1f33df6b d655d8e6`	W_8–W_{15}
	`15730eaa 9bf64950 9f129973 5a964edf`	
P3	`1bd7ebbd c4d938fb 26d19b5d d5caf333`	81.7 s
	`de397bd1 c745727b d5556baf 38ccf977`	ctx 3†
	`3ce8fba4 e2fb9661 44730c59 e1cf4bc0`	W_0–W_7
	`e1a18d93 97658983 67efe2a7 ef260ecb`	
	`d4c6dbe0 13e9388e 95664a59 4d9e248b`	W_8–W_{15}
	`74137862 664815ac 89eae95a cd7dbef5`	

6.3 Chi-Squared Uniformity Test

To test the $1/2^{16}$ model for the W9-hi filter, we collected the values $(W_9^{\text{true}} \gg 16) \bmod 2^{16}$ for all lo-pass events across 165 GPU contexts (a total of 5,775 lo-pass events). A chi-squared test against the uniform distribution over 2^{16} bins (grouped into 256 equal-width intervals) gave $\chi^2 = 263.4$ with 255 degrees of freedom ($p = 0.35$).

This test is *not* sensitive to the effect that matters, and we record the point because it is easy to be misled by it. The enhancement is concentrated at a single residual value out of 2^{16}, whereas grouping into 256 equal-width intervals spreads that single value across a bin of width 256 and dilutes the signal by roughly an order of magnitude. We measured the dilution directly on the *lo* residual, where the exact-value enhancement is known: a 37× spike at the exact residual appears as only 3.9× at 256-bin resolution. A comparable hi enhancement would therefore shift the occupancy of one bin from 22.6 to roughly 32—entirely invisible to a χ^2 statistic over 255 degrees of freedom.

Measuring at exact resolution instead shows the hi filter passing at $\approx 9.2\times$ the uniform rate (§5.6). The correct conclusion is that the coarse-binned test is underpowered, not that the hi residuals are uniform; filter rates in this attack must be estimated at exact residual resolution.

7 Discussion: R=20 Status

The backward chain at $R = 20$ recovers $a_{12}, \ldots, a_{19}$ but not a_{11}. The schedule grows to four constraints W_{16}–W_{19}, with a_{11} as an additional unknown.

Oracle-conditioned result. When the context including a_{11} is supplied by a known synthetic pair, the W_{17} identity gives a_2 as an explicit function of a_3 (Lemma analogous to Proposition 2), and an h-table $h[W_{12}(a_{11})] \to a_{11}$ enables an $O(2^{32})$ a_3 sweep. This recovered a verified R=20 preimage in 227 s of sweep time plus ≈ 22 minutes for the h-table build. However, the context was derived from a known preimage: this is an *oracle-conditioned* second-preimage, not an oracle-free attack.

Why oracle-free R=20 is not established. The h-table inversion requires $W_{12}^{\text{req}} = W_{19}^{\text{bc}} -$

$\sigma_1(W_{17}^{\text{bc}}) - \sigma_0(W_4) - W_3$, where W_{17}^{bc} carries a_{11} through e_{13} (which contains $\text{Maj}(a_{12}, a_{11}, a_{10})$). Thus a_{11} is needed to compute the quantity used to look up a_{11}: a circular dependency. No algebraic link that breaks this circularity has been found. Exhaustive search over all binary linear combinations of the four residual equations R_{16}–R_{19} found no combination that isolates any of the three unknowns (a_2, a_3, a_{11}).

Quantitative complexity gap. An equivalent view using the σ_0-chain directly confirms the 2^{32} difficulty increase. Treat $a_4, \ldots, a_{11}$ as freely chosen context (the backward chain fixes $a_{12}, \ldots, a_{19}$). The chain C0–C2 then proceeds identically to the $R = 19$ case, but the schedule now includes W_{19}, introducing a fourth constraint (C3):

$$\sigma_0\big(W_4^{\text{round}}\big) = W_{19}^{\text{bc}} - \sigma_1(W_{17}^{\text{bc}}) - W_{12}^{\text{base}} - W_3,$$

where W_4^{round} is fully determined by $a_0, \ldots, a_3$ once C0–C2 converge and W_{12}^{base} is determined by context. Because σ_0 is a bijection, W_4^{round} must equal the unique $\sigma_0^{-1}(C3_{\text{tgt}})$. C3 therefore contributes an additional 32-bit equality. Under the present construction we model it as approximately uniform and independent of C0–C2, so that it passes with probability $\approx 2^{-32}$; we stress that this is a statistical model which has not been established directly at full width.

Exhaustive experiments in a reduced-width scale model (§7.1) bear on the model but do not confirm it at $w = 32$. Measuring $P(\text{C3} \mid \text{C0}, \text{C1}, \text{C2})$ against the independent prediction 2^{-w} gives a ratio of 1.075 ± 0.018 at $w = 4$ and 1.005 ± 0.035 at $w = 5$: consistent with exact independence at the larger width, with a small excess at the smaller one that may be a narrow-width artefact. Any dependence is thus small, but two widths do not establish a trend, and the behaviour at $w = 32$ remains untested.

Schedule identity. For messages satisfying the σ_0-chain $W_k = \sigma_0(W_{k+1})$, $k = 0, \ldots, 3$, the substitution $\sigma_0(W_{r-15}) = W_{r-16}$ reduces the schedule recurrence for $r \in \{16, \ldots, 19\}$ to

$$W_r = \sigma_1(W_{r-2}) + W_{r-7} + 2\,W_{r-16}.$$

From this identity and $W_3 = \sigma_0(W_4)$ one computes directly $C3_{\text{tgt}} = W_{19} - \sigma_1(W_{17}) - W_{12} - W_3 = \sigma_0(W_4) = W_3$, confirming that the C3 check reduces to $W_4^{\text{round}} = W_4^{\text{chain}}$ (verified numerically). The same identity provides a concise derivation of the Nine-Step Lemma: at $r = 16$, $\Delta W_{16} = \Delta W_9 + \Delta W_0 = -\delta + \delta = 0$, and the doubled term $2W_{r-16}$ ensures that *any* perturbation $\Delta W_1 = \varepsilon$ would introduce $\sigma_0(\varepsilon)$ into ΔW_{16} (breaking cancellation), which is why the pair $(0, 9)$ is the unique minimal solution.

Orbit structure of σ_0. Viewed as a $\mathbb{F}_2$-linear bijection on $(\mathbb{Z}/2\mathbb{Z})^{32}$, the map σ_0 has order $49{,}146 = 2 \cdot 3 \cdot 8191$ where $8191 = 2^{13} - 1$ is a Mersenne prime. Its only short orbits are the two fixed points $\{0, \texttt{0x27f42515}\}$ and the unique 2-cycle $\{\texttt{0x0b3b7ef9} \leftrightarrow \texttt{0x2ccf5bec}\}$; in particular, no period-3 or period-4 elements exist. Consequently any σ_0-chain of length four $(W_0, \ldots, W_4)$ lies in an orbit of length 49,146 unless W_4 is one of the four exceptional values above, so no "chain-wrap" shortcut of the form $W_0 = W_4$ arises generically.

Under the variable ordering and single-representative lookup construction used here, C3 remains an additional 32-bit consistency condition applied after C0–C2 have been solved. We emphasise what is and is not established. The algebra shows C3 imposes a further 32-bit equality; it does not show that every algebraic arrangement must pay 2^{32} for it. No absorption of C3 into the construction, and no lower-width decomposition of it, is currently known—but the negative results below are properties of the parameterisations we tried, not proofs of impossibility. Indeed one apparent obstruction at R=20, the $a_1 \leftrightarrow a_2$ cycle, dissolved entirely under a change of variable (§7.2), which is direct evidence that such obstructions can be artefacts of representation. Treating C3 as an independent filter, the cost is: Using the measured per-context success rate the projected $\mu_{R=19} \approx 6 \times 10^{-3}$ (§5.6), the R=20 expected contexts is $\mu_{R=19}^{-1} \times 2^{32} \approx 170 \times 2^{32} \approx 2^{39.4}$—a factor of 2^{32} harder than R=19. The ratio is what matters here and is independent of the value of $\mu_{R=19}$; only the absolute figure moves with it, and that figure inherits the projection's caveats.

GPU diagnostic. A 165-context GPU run of the $R = 20$ σ_0-chain on an H100 (batch $= 2^{26}$, $K_{seeds} = 4$) recorded 4,166 W_9-lo-pass events and zero hi-pass events. Under the null hypothesis of uniform residuals, the expected hi count is $\mu = 4166/65536 \approx 0.064$; the probability of zero hi events is $e^{-\mu} \approx 0.94$, so the absence of hi events is unsurprising. With an expectation below 0.1, however, observing zero carries almost no information: the result is *consistent with* a uniform 2^{-32} filter but does not test it. The observed difficulty increase over $R = 19$ is attributable to the additional C3 equality rather than to any structural bias in W_9; the attribution of the full 2^{32} factor to C3 rests on the uniformity model stated above, which this run is too small to test. A supplementary CPU sweep (10 contexts, batch $= 2^{24}$, two workers) recorded 254 W_9-lo-pass events and zero C3-pass events, yielding an enhancement upper bound of $2^{26.3}$ (95%). This too is consistent with a uniform filter and far too small a sample to test independence or estimate its probability. The structural claim—that C3 imposes an additional 32-bit equality—rests on the algebra, not on these counts; the probabilistic claim should be read as untested.

7.1 A Reduced-Width Scale Model

Claims about C3's independence, and about whether the chained solver is lossy, are untestable at full width: the events are too rare to sample and the solution set cannot be enumerated. We therefore build a scale model of the round function and message schedule at word width w, where the inner space $(a_0, a_1, a_2, a_3, a_{11})$ is small enough to enumerate exhaustively. Rotation and shift amounts are scaled from SHA-256's by $w/32$ and rounded; K and the IV are truncated. The construction preserves the shape of the mixing and is not proposed as a standard.

Two fidelity checks are passed. At $w = 32$ the scaled constants reduce to SHA-256's exactly, and the model reproduces SHA-256's state evolution word for word. The solution count per context matches the degrees-of-freedom prediction: five unknown words less four constraints leaves one surplus word, hence 2^w solutions, against 19.2 measured at $w = 4$ ($2^4 = 16$) and 32.0 at $w = 5$ ($2^5 = 32$).

The independence measurement is reported in §7. The model also shows that the chained fixed-point architecture is intrinsically lossy rather than merely slow. A solver using one representative per table entry recovers 1.3%, 0% and 0% of the true solutions at $w = 4, 5, 6$, while one branching over all roots recovers 23.4%, 6.8% and 0.7%. Raising the beam from 64 to 1024 and the iteration count from 6 to 20 changed coverage not at all, so the loss is structural rather than a truncation effect. Tables should therefore be relations, not functions. This monotone collapse, resting on 36, 13 and 2 recovered solutions, is the model's robust finding.

We deliberately do *not* extrapolate these rates to $w = 32$. The per-a_0 success rate falls by $4.15\times$ from $w = 4$ to $w = 5$ and by $8.67\times$ from $w = 5$ to $w = 6$, so the point estimates suggest an accelerating rather than log-linear decay; but the $w = 6$ figure rests on two events, whose exact Poisson interval puts the second ratio anywhere in $[2.40\times, 71.6\times]$, a range that contains the first. The data therefore constrain neither the functional form nor the rate, while a 5% change in the fitted rate moves a 27-bit extrapolation by a factor of 4.5. We note additionally that an earlier version of this model, containing an initial-state indexing error, produced an apparent agreement with our full-width bound that did not survive correction—a cautionary instance of how little such an extrapolation constrains. The scale model's value lies in its exhaustive in-range comparisons, not in extrapolated rates.

7.2 Absorber Reparameterisation and the $a_1 \leftrightarrow a_2$ Coupling

The four residual equations can each be written in the $\sigma_0(u) - u$ form used by the R=19 table. Substituting the round-function expression $W_{10} = \mathrm{CONST}_{10} - d(a_3) - a_2$ into the W_{17} identity and then eliminating a_2 via $a_2 = W_2 - F_{12}(a_1)$ yields:

Lemma 2 (Reparameterisation of C1)**.** *Write* $A(a_1) := a_1 + gv + F_{12}(a_1)$*, a function of* a_1 *alone. Then*

$$\sigma_0(W_2) - W_2 \;=\; h_1(a_{11}) - \mathrm{CONST}_{10} + d(a_3) - A(a_1) \;=:\; u,$$

so the target u *decomposes additively into independent contributions from* a_{11}*,* a_3 *and* a_1*.*

Verified exactly on 2000/2000 random triples (a_1, a_3, a_{11}). Taking u as the free variable inverts the map: W_2 is one table lookup, $a_1 = A^{-1}(K_1 - u)$ a second, and $a_2 = W_2 - F_{12}(a_1)$. Hence for fixed (a_3, a_{11}) each u determines *both* a_1 and a_2 in $O(1)$. The apparent circular dependency between C0 (which needs a_2) and C1 (which needs a_1) is therefore an artefact of the parameterisation, not a genuine fixed-point problem; this explains why naive iteration of that pair never converges. (A is a near-bijection: 3 collisions in 2×10^5 samples, so A^{-1} requires a bucketed table.)

Two further reductions place the remaining constraints in table form. In C2 the unknown a_3 cancels from its own target, since $e_7 = a_3 + T1_7$ and $W_3 = a_3 + f_{23}$:

$$\sigma_0(W_3) - W_3 = \big(W_{18} - \sigma_1(W_{16}) - W_{11}\big) - W_2 - f_{23} + T1_7.$$

In C3, the quantities e_{16}, e_{17}, e_{18} do not depend on a_{11} (they are fixed by the backward chain), so $\sigma_1(e_{18})$ and $\mathrm{Ch}(e_{18}, e_{17}, e_{16})$ are per-context constants and

$$W_{19} \;=\; \mathrm{CONST}_{19} - \big(a_{11} + T1_{15}\big)$$

is *linear* in a_{11}; only W_{17} and W_{12} carry nonlinear a_{11}-dependence into the C3 key.

7.3 A Context Family that Removes a_2 from C0

The variable a_2 reaches the C0 target by exactly two routes: the term $\mathrm{Ch}(e_8, e_7, e_6)$ with $e_6 = a_2 + t_{16}$, and $\mathrm{Maj}(a_4, a_3, a_2)$ inside t_{15}. Both are maskable by context choice. Setting $e_8 =$ `0xFFFFFFFF` (reachable by choosing a_8, since $e_8 = a_4 + a_8 - \Sigma_0(a_7) - \mathrm{Maj}(a_7, a_6, a_5)$) collapses $\mathrm{Ch}(e_8, e_7, e_6)$ to e_7; setting $a_4 = a_3$ collapses $\mathrm{Maj}(a_4, a_3, a_2)$ to a_3. Measured over 500 random a_2 per trial, the number of distinct C0 targets is:

context condition	distinct C0 targets
random context	400/400
$e_8 =$ `0xFFFFFFFF` alone	400/400
$a_4 = a_3$ alone	217, 489
$a_4 = a_3$ *and* $e_8 =$ `0xFFFFFFFF`	1, 1, 1, 1

Neither condition suffices alone; together they eliminate a_2 from C0 entirely, turning it into a two-input absorber $(a_3, a_{11}) \mapsto a_1$. This is the first demonstration that the R=20 constraint *structure* is shapeable by context choice rather than fixed. It does not by itself reduce the constraint count.

7.4 The R=19 Iteration Architecture Does Not Transfer Directly

The R=19 attack does not schedule its constraints acyclically: it too has a cycle (C0 requires W_9, which requires a_2, a_3, which come from C1/C2, which require a_1 from C0). It breaks the cycle by freezing W_9 at a provisional value, iterating C1↔C2 to a fixed point, and paying one 32-bit self-consistency check. Convergence of that iteration is what satisfies C1 and C2.

Promoting a_{11} from guessed context to an unknown absorbed by C3 gives R=20 the same one-word surplus (5 unknowns $a_0..a_3, a_{11}$ against 4 constraints, versus R=19's 4 against 3), and we verified that the true R=20 solution is a fixed point of all four absorber maps. The architecture therefore transplants in form. It does not transplant in efficacy. Measured with one harness, cold start, 8 iterations, no seeding, on the same hardware:

iteration	convergences / trials	rate per a_0
R=19 control (two-map, C1↔C2)	1,790 / 33,554,432	5.34×10^{-5}
R=20 (four-map, C0–C3)	0 / 1,996,488,704	$< 1.50 \times 10^{-9}$ (95%)

The R=19 control reproduces the documented rate band, so the harness is sound and the R=20 null is a measurement rather than an artefact. In iteration-steps per preimage the classic sweep costs 2^{64} while the iterated variant costs $2^{35}/p$, so the break-even convergence rate is $p = 2^{-29} \approx 1.86 \times 10^{-9}$; the measured bound lies below it. We note the margin is modest—the 95% bound is a factor 1.24 under break-even—so the conclusion is that the iterated variant does not beat the classic sweep, not that it loses by a wide factor; a longer run would tighten the bound proportionally. The structural estimate is far more pessimistic than the bound: R=19's 5.34×10^{-5} sits about 2^{15} above the 2^{-29} random-map baseline for a 32-bit state, and carrying that same enhancement to a 64-bit state (baseline $\approx 2^{-61}$) predicts $p \approx 2^{-46}$, some five orders of magnitude below break-even. The cause is dimensional: at R=19 the absorbed a_1 is computed *outside* the loop, leaving a 32-bit iterated state (a_3, with a_2 derived), whereas at R=20 C0 depends on a_{11} and the state becomes 64-bit (a_3, a_{11}). Raising the iteration budget does not help, since convergence probability and cost scale alike.

7.5 A Second Global Table: the a_0 Absorber

The attack sweeps a_0 and uses C0 to absorb a_1 through the global $\sigma_0(u) - u$ table. The two roles can be exchanged. Writing gv for the round-1 constant and using

$$W_1 = a_1 + gv(a_0), \qquad W_0 = a_0 + \mathrm{C0}_{\mathrm{CONST}}, \qquad W_9(a_1) = W_9^{(0)} - a_1,$$

where $W_9^{(0)}$ is W_9 evaluated at $a_1 = 0$, the constraint $W_{16} = \sigma_1(W_{14}) + W_9 + \sigma_0(W_1) + W_0$ rearranges to give:

Proposition 3. *Let* $\Gamma(a_0) := a_0 + gv(a_0)$ *and* $R := W_{16} - \sigma_1(W_{14}) - W_9^{(0)} - \mathrm{C0}_{\mathrm{CONST}}$. *Then C0 is equivalent to*

$$\Gamma(a_0) \;=\; R + W_1 - \sigma_0(W_1).$$

The point is that gv depends only on a_0 and the SHA-256 IV—no context word and no target enters it—so Γ is a fixed function and Γ^{-1} is a *global* table, built once and valid for every target and context, exactly like $\sigma_0(u) - u$. Measured over all 2^{32} inputs, Γ covers 63.214% of the range against $1 - e^{-1} = 63.212\%$ for a random map, so the table behaves like the one already in use. Proposition 3 is verified on all three published preimages by `code/verify_a0_absorber.py`, which also confirms it is not a tautology: perturbing W_{16} by $1, \ldots, 64$ breaks it in every case.

This does not reduce the attack's cost, and the reason is worth stating because it sharpens the $R = 20$ barrier. C0 can absorb a_0 *or* a_1, but not both. With five unknowns $(a_0, a_1, a_2, a_3, a_{11})$ and four constraints, and with a_2 absorbable only by C1, a_3 only by C2 and a_{11} only by C3, the two variables a_0 and a_1 compete for the single constraint C0. One of them must therefore be swept, and whichever four remain form a cycle in which every absorber requires three of the others. That competition, rather than any property of a particular variable ordering, is what forces two guessed words at $R = 20$.

7.6 A Linear Identity for σ_0, and its Destruction

Since σ_0 is $\mathbb{F}_2$-linear, put $L = \sigma_0 \oplus I$. The kernel of L is the fixed-point set $\{0, \texttt{0x27f42515}\}$, so $\operatorname{rank} L = 31$ and the image is a 31-dimensional subspace. Consequently there is a *unique* nonzero mask annihilating it.

Proposition 4. *Let* $\lambda = \texttt{0xa8a42fe4}$. *Then for every* $W \in (\mathbb{Z}/2\mathbb{Z})^{32}$,

$$\lambda \cdot \big(\sigma_0(W) \oplus W\big) = 0.$$

Verified exhaustively over all 2^{32} inputs (zero violations). This is the left companion of the known right-kernel constant `0x27f42515`.

The identity is however destroyed by modular subtraction, which is what the attack actually uses. Exhaustively over all 2^{32} inputs,

$$\Pr_W\big[\lambda \cdot (\sigma_0(W) - W) = 1\big] = 0.500017190,$$

a bias of $1.72 \times 10^{-5} = 2^{-15.8}$, against an exact 0 for the XOR form. A relation that is identically zero on the $\mathbb{F}_2$-linear part survives modular subtraction only as a $2^{-15.8}$ bias, which affords no usable filtering. Together with the table coverage—both the σ_0 table and the C3 h-table measure 63.21% against $1 - e^{-1} = 63.212\%$, matching a random map to four significant figures—the map $W \mapsto \sigma_0(W) - W$ is indistinguishable from a random map under every structural test we have applied. The interaction between $\mathbb{F}_2$-linear σ_0 and modular arithmetic is thus not merely an obstacle to proof technique; it is the mechanism by which R=20 resists.

The oracle-free R=20 attack remains an open problem.

8 The Nine-Step Cancellation Lemma and Collision Structure

8.1 Nine-Step Cancellation Lemma

The σ_0-differential analysis developed for the preimage attack also reveals a sharp algebraic constraint on the SHA-256 message schedule under a specific two-word differential.

Lemma 3 (Nine-Step Cancellation)**.** *Let $W_0, \ldots, W_{15}$ be any message block and let $\delta \neq 0$. Define $W'_i = W_i$ for $i \notin \{0, 9\}$, $W'_0 = W_0 + \delta$, and $W'_9 = W_9 - \delta$ (all arithmetic mod 2^{32}). Then the extended schedules satisfy*

$$W'_r - W_r = 0 \pmod{2^{32}}, \quad r = 16, 17, \ldots, 23.$$

Furthermore, $W'_{25} - W_{25} = -\delta$ exactly (for all W, all δ).

Proof. The schedule recurrence is $W_r = \sigma_1(W_{r-2}) + W_{r-7} + \sigma_0(W_{r-15}) + W_{r-16}$. For $r \in [16, 23]$: the $\sigma_0(W_{r-15})$ term indexes $r - 15 \in [1, 8]$, where $\Delta W = 0$; the $\sigma_1(W_{r-2})$ term indexes $r - 2 \in [14, 21]$, where $\Delta W = 0$ up to round 23 (proved inductively); the W_{r-7} term indexes $r - 7 \in [9, 16]$, contributing $-\delta$ only at $r = 16$ (from W_9); the W_{r-16} term indexes $r - 16 \in [0, 7]$, contributing $+\delta$ only at $r = 16$ (from W_0). At $r = 16$: $\Delta W_{16} = 0 + (-\delta) + 0 + \delta = 0$. For $r \in [17, 23]$: all four terms contribute zero, so $\Delta W_r = 0$. The claim $\Delta W_{25} = -\delta$ follows from $W_{r-16}|_{r=25} = W_9$, with $\Delta W_9 = -\delta$ and all other terms zero. □

The pair $\{0, 9\}$ is the *unique* two-position pair in $[0, 15]^2$ achieving a full algebraic dead zone. Two necessary conditions are:

1. σ_0 *silence:* The $\sigma_0(W_k)$ term enters round r when $r - 15 = k$. For no σ_0 contribution to land *inside* $[16, 23]$, both i and j must satisfy $k + 15 \leq 15$ (i.e. $k = 0$, landing at $r = 15$, *before* the window) or $k + 15 \geq 24$ (i.e. $k \geq 9$, *after* the window). Among $[0, 15]$ the valid positions are 0 and $\{9, 10, \ldots, 15\}$. A pair must draw both positions from this set.

2. *Linear cancellation at $r = 16$:* W_{16} has linear terms W_9 and W_0. Any nonzero ΔW_k with $k \in \{0, 9\}$ must cancel: $\Delta W_0 + \Delta W_9 = 0$ at $r = 16$.

Condition 1 forces the pair into $\{0\} \cup [9, 15]$. Condition 2 then requires exactly one position to be 0 and one to be 9 (for the W_0 and W_9 linear contributions to cancel). Any pair with both positions in $[9, 15]$ leaves $\Delta W_{16} = \Delta W_j \neq 0$ uncancelled; any pair with both positions $= 0$ is degenerate. Hence $(0, 9)$ (and its sign-reversal $(9, 0)$) are the unique solutions.

For partial dead zones, choosing the minus-delta position $j < 9$ causes $\sigma_0(W_j)$ to land at $W_{j+15} \in [16, 23]$—a nonzero "seed" that propagates to alternating positions thereafter. The first nonzero schedule difference is at W_{j+15}, so pairs with $j \in \{4, 7, 8\}$ produce 5 or 6 dead positions in $[16, 23]$, which is the basis of the $(13, 4)$ and $(7, 8)$ results below.

TC extension. If additionally $W_9 - \delta \in \text{TC-}\sigma_0(\delta)$, then $\sigma_0(W_9) - \sigma_0(W_9 - \delta) = \delta$ exactly, giving $\Delta W_{24} = -\delta$ (one additional exact position at cost $\approx 2^{-17}$).

8.2 State-Differential Trap and Boomerang Structure

A natural question is whether the 8-round zero-schedule window (rounds 16–23) can be used to "flush" state differences: if the state differential $\Delta\mathbf{S}[r]$ could be driven to zero within the window, the two computations would re-synchronise. We show this is impossible.

Proposition 5 (State-Differential Trap)**.** *Let $\Delta W_r = 0$ for all r in some range. Then $\Delta\mathbf{S}[r + 1] = \mathbf{0}$ if and only if $\Delta\mathbf{S}[r] = \mathbf{0}$.*

Proof. The shift terms of the round function force $\Delta b[r + 1] = \Delta a[r]$, $\Delta c[r + 1] = \Delta b[r]$, …, $\Delta h[r + 1] = \Delta g[r]$. So $\Delta\mathbf{S}[r + 1] = \mathbf{0}$ requires $\Delta a[r] = \cdots = \Delta g[r] = 0$. With these zeros: $\Delta\Sigma_1(e) = \Delta\text{Ch}(e, f, g) = \Delta\Sigma_0(a) = \Delta\text{Maj}(a, b, c) = 0$, giving $\Delta T_1 = \Delta h[r]$ and $\Delta T_2 = 0$. The Δe-equation then forces $\Delta h[r] = 0$. Hence all eight words of $\Delta\mathbf{S}[r]$ are zero, i.e. $\Delta\mathbf{S}[r] = \mathbf{0}$. □

Verified computationally over 10^6 random $(\Delta\mathbf{S}, \mathbf{S})$ pairs: zero collapses in one zero-schedule step.

The correct interpretation of the zero window is as a *deterministic boomerang connector* [8]: the 8-round function E_{mid} maps $\Delta\mathbf{S}[16] \mapsto \Delta\mathbf{S}[24]$ with *probability one*, eliminating eight rounds of schedule probability conditions from any differential trail that passes through it. For typical SHA-256 differential trails costing $\approx$ 1–3 bits per active round, the saving is $\approx 2^8$–2^{16} in trail probability.

8.3 GPU Sweep and One-Word Partial Collision

Using the Nine-Step differential $\Delta W_0 = +\delta$, $\Delta W_9 = -\delta$ with $W_9 - \delta \in \text{TC-}\sigma_0$, we fixed W_0=`0x6ac550a3`, W_9=`0x001c2641`, $W_{10..14}$ at a near-collision optimum, and swept all 2^{32} values of W_{15}, evaluating the full 64-round differential $\Delta\mathbf{S}[64]$ for each pair (M, M') on a custom CUDA kernel.

GPU performance. On an NVIDIA RTX 4060 (custom CUDA C kernel via CuPy, CUDA 12.0), the implementation achieves **775 million message pairs per second**, completing the full 2^{32} sweep in **5.5 seconds**. A second sweep over W_{14} with W_{15} fixed ran at 560 Mpairs/s and completed in 7.7 seconds.

One-word partial collision (Theorem). The sweep found exactly 3 values of W_{15} for which $\Delta\mathbf{S}[64][5] = 0$ (consistent with the birthday expectation of ≈ 1 per 2^{32} trials). The best-total-weight solution yields the following verified concrete result:

Theorem 1 (One-Word Partial Collision)**.** *Let δ = `0x0011e034`. Under the Nine-Step differential ($\Delta W_0 = +\delta$, $\Delta W_9 = -\delta$), the message pair*

$$M = [\texttt{0x6ac550a3}, \ldots, \texttt{0xa56abdfe}, \texttt{0x82627b41}],$$
$$M' = [\texttt{0x6ad730d7}, \ldots, \texttt{0xa56abdfe}, \texttt{0x82627b41}] \quad (\textit{only } W_0, W_9 \textit{ differ})$$

satisfies

$$\text{SHA-256\_compress(IV, } M)[5] \;=\; \text{SHA-256\_compress(IV, } M')[5] \;=\; \texttt{0x3adcd2d0}.$$

Equivalently, the raw post-round working-state word before IV feed-forward is `0x9fd76a44` for both computations. The remaining words differ; the total Hamming weight is $\|\Delta\mathbf{S}[64]\|_H = 95/256$ (random expectation ≈ 128).

The complete $\Delta\mathbf{S}[64]$ vector ($\star$ = zero word) is:

$$[\texttt{0xc85da50e},\ \texttt{0xca8164c8},\ \texttt{0x0ac720e6},\ \texttt{0x94ad1918},$$
$$\texttt{0x87038da6},\ \underbrace{\texttt{0x00000000}}_{\star},\ \texttt{0x78d3023a},\ \texttt{0x468338d3}].$$

Full enumeration. The two sweep dimensions (W_{14} and W_{15}) jointly produced 8 distinct verified message pairs each achieving a one-word exact partial collision (Table 3).

Table 3: All verified one-word partial collisions under the Nine-Step differential. "Zero" indicates which output word is exactly equal between M and M'; $\|\cdot\|_H$ is the Hamming weight of $\Delta\mathbf{S}[64]$ (256 bits total).

W_{14}	W_{15}	Zero word	$\|\Delta\mathbf{S}[64]\|_H$
0xa56abdfe	0x82627b41	5	95
0xa56abdfe	0x5c7feda3	5	105
0xa56abdfe	0xc360541e	5	111
0x316eef2c	0x82627b41	5	108
0x128f99b2	0x82627b41	4	117
0xc26dd92f	0x82627b41	4	109
0xc26dd92f	0x0b6abb05	4	111
0xc26dd92f	0x3de7f8eb	4	120

All 8 results are independently verified by `final_results.py` using the reference SHA-256 implementation. The schedule structure is confirmed: $\Delta W_{16..23} = \mathbf{0}$ (exact, algebraic), $\Delta W_{25} = -\delta$ (exact).

Two-word partial collision. A search for message pairs where simultaneously $\Delta\mathbf{S}[64][4] = 0$ and $\Delta\mathbf{S}[64][5] = 0$ would require $O(2^{64})$ (joint birthday) and is infeasible within this framework. Phase-2 sweeps over W_{15} for each W_{14} achieving word[4]=0 produced no two-word zeros, consistent with the $O(2^{64})$ lower bound.

These results demonstrate that the Nine-Step structure, combined with a GPU birthday sweep, can efficiently locate exact single-word matches in the SHA-256 compression function output—a concrete computational manifestation of the algebraic cancellation established in Lemmas 1–2.

Multi-round near-collision profile. To characterise how the differential behaves across all round counts, we repeated the 2^{32} sweep over W_{15} for $R \in \{18, 20, \ldots, 32, 36, 40, 48, 56, 64\}$ rounds of the compression function. Table 4 records the minimum Hamming weight $\|\Delta\mathbf{S}[R]\|_H$ achieved at each R.

Three observations emerge. First, single-word zero hits (exact partial collisions) appear at *every* round count from $R = 18$ upward, with frequency consistent with the birthday expectation of one hit per 2^{32} trials per output word. Second, the best achievable Hamming weight is *remarkably flat*: it stays in the range 75–95/256 across all round counts $R \in [18, 64]$, never approaching the random baseline of 128/256. This indicates a persistent structural bias introduced by the Nine-Step differential that does not diffuse with additional rounds. Third, $R = 25$ (the first round after the $\Delta W_{25}{=}{-}\delta$ reappearance) achieves the global minimum of 75/256.

Two-block near-collision (coordinate descent). The State-Differential Trap (Proposition 5) shows that a full single-block collision would require $O(2^{64})$ work. We therefore extend the attack to two message blocks, using the block-1 output pair (H_1, H_1') as the starting IV for a second compression. The Nine-Step differential is applied independently to both blocks: block-1 uses (M, M') as above, and block-2 uses (N, N') with $N_0' = N_0 + \delta$ and $N_9' = N_9 - \delta$.

To find a block-2 message $N[0..15]$ minimising the final Hamming weight $\|\Delta H_2\|_H = \|\mathrm{CF}(H_1', N') - \mathrm{CF}(H_1, N)\|_H$, we apply *coordinate descent*: iterate over each of the 16 message words, sweeping that word over all 2^{32} values (full exhaustive search) while holding the

Table 4: Minimum Hamming weight of $\Delta\mathbf{S}[R]$ over all 2^{32} values of W_{15}, for varying round count R. The random expectation is $\approx 128/256$ at every R. Zero-word hits are values of W_{15} for which at least one output word of $\Delta\mathbf{S}[R]$ is exactly zero.

R	Best $\|\Delta\mathbf{S}[R]\|_H$	Random exp.	Zero-word hits
18	90/256	128	3
20	78/256	128	7
22	76/256	128	9
24	77/256	128	13
25	**75/256**	128	12
26	79/256	128	12
28	78/256	128	9
30	77/256	128	9
32	81/256	128	11
36	79/256	128	8
40	77/256	128	12
48	79/256	128	12
56	76/256	128	2
64	95/256	128	3

other 15 fixed, then accept the minimising value. Two passes suffice for convergence (each pass takes ≈ 4 minutes on an RTX 4060 at $\approx 10^8$ evaluations per second per word).

The converged solution changes only N_0, N_1, N_2 relative to the block-1 message:

$$N_0 = \texttt{0x254b45f2}, \quad N_1 = \texttt{0xae6bc5c0}, \quad N_2 = \texttt{0x531ffc39},$$
$$N_i = M_i \quad (i = 3, \ldots, 15).$$

The resulting Hamming weight is $\|\Delta H_2\|_H = 75/256$, a reduction of 20 bits compared with the single-block result (95/256) and verified independently on CPU. The per-word decomposition is:

$$\Delta H_2 = [\texttt{0x00293948},\ \texttt{0xa15a1b48},\ \texttt{0xa4433504},\ \texttt{0x081c0024},$$
$$\texttt{0x8885f682},\ \texttt{0x01107096},\ \texttt{0x2a108002},\ \texttt{0xac210a00}],$$

with individual word contributions of $9, 13, 11, 6, 13, 9, 6, 8$ bits. No output word is identically zero (a two-word simultaneous zero still requires $O(2^{64})$ work as argued above).

Alternative differentials. An exhaustive algebraic survey of all 240 two-position message-schedule differentials ($\Delta W_i = +\delta,\ \Delta W_j = -\delta$) classifies each pair by how many of $W_{16}, \ldots, W_{23}$ are algebraically zero (for all messages and all δ). Only $(i, j) = (0, 9)$ and $(9, 0)$ achieve the full 8-round dead zone. Two pairs — $(7, 8)$ and $(8, 7)$ — achieve a 6/8 dead zone (first six positions: W_{16}–W_{21}). Sixteen further pairs achieve a 5/8 dead zone, grouped into four structural patterns (Table 5).

All 10 pairs with $\geq 5/8$ dead positions and $|\Delta H_1| \leq 120/256$ were analysed via block-2 coordinate descent (marked † in Table 5). The $(3, 12)$ differential achieves **72/256**—matching the three-block Nine-Step minimum in just two blocks. The $(11, 13)$ pair achieves 73/256, matching the $(12, 3)$ reversed result. The $(13, 4)$ and $(13, 7)$ pairs each achieve 74/256. The Nine-Step pairs $(0, 9)$, $(9, 0)$ and the $(7, 8)$ pair achieve 75/256.

The exact pattern of zero positions is algebraically determined: for $(13, 4)$ the five zero positions are $W_{16}, W_{17}, W_{18}, W_{20}, W_{22}$, and the three nonzero positions are W_{19}, W_{21}, W_{23} (alternating, for all messages and all δ). The nonzero terms arise from $\sigma_0(W_4)$ applied to the modified word, which first enters at round 19:

$$W_{19} = \sigma_1(W_{17}) + W_{12} + \sigma_0(W_4) + W_3.$$

Table 5: Algebraic dead zone patterns for the 20 pairs with $\geq 5/8$ dead positions in $W_{16..23}$. "1" = algebraically zero, "0" = nonzero (for generic message). †Two-block coord descent run and reported in Table 6.

Pattern ($W_{16..23}$)	Dead	Representative pairs	$\|\Delta H_2\|_H$
11111111	8/8	$(0,9)^\dagger$, $(9,0)^\dagger$	75/256
11111100	6/8	$(7,8)^\dagger$, $(8,7)^\dagger$	see Tab. 6
11111000	5/8	$(6,7)$, $(6,8)$, $(7,6)$, $(8,6)$	—
11110100	5/8	$(7,13)$, $(8,13)$, $(13,7)^\dagger$, $(13,8)$	74/256 for $(13,7)$
11101010	5/8	$(4,13)^\dagger$, $(8,12)$, $(12,8)$, $(13,4)^\dagger$	74–76/256
11010101	5/8	$(3,12)^\dagger$, $(11,13)^\dagger$, $(12,3)^\dagger$, $(13,11)$	see Tab. 6

The even-indexed positions W_{20} and W_{22} remain zero because their $\pm\delta$ contributions cancel linearly ($\Delta W_{20} = +\delta - \delta = 0$ exactly). Compared to the Nine-Step, the (13,4) residual is *scattered* rather than a single "wall" appearing at W_{24}, allowing the compression function to suppress the residual difference more efficiently. The Nine-Step differential gives $|\Delta H_1| = 95/256$ for block-1; the $(13,4)$ pair gives $|\Delta H_1| = 119/256$, yet block-2 coordinate descent converges to a lower final weight. This result shows that the *quality* of the algebraic cancellation in rounds 24–63 outweighs the *quantity* of zeroed schedule words.

Schedule Structure Lemma. The alternating nonzero pattern of $(13,4)$ is an instance of a general algebraic structure that governs all $|p-q| = 9$ differentials.

Lemma 4 (Schedule Density)**.** *Let* ($\Delta W_p = +\delta$, $\Delta W_q = -\delta$) *with* $|p-q| = 9$ *and* $\min(p,q) < 9$*, so that* $r_0 = \min(p,q) + 15 \in [16,23]$*. For all messages and all* $\delta \neq 0$*:*

1. Interleaved zeros (exact): $\Delta W_{r_0+1} = \Delta W_{r_0+3} = \Delta W_{r_0+5} = 0$.
2. Period-7 resonance (exact): $\Delta W_{r_0+7} = \Delta W_{r_0}$ *(equal, not necessarily zero).*

Proof. Interleaved zeros: At $r = r_0 + 1$: $W_{r-7} = W_{\max(p,q)}$ contributes $-\varepsilon\,\delta$ and $W_{r-16} = W_{\min(p,q)}$ contributes $+\varepsilon\,\delta$ (where $\varepsilon\,\delta$ is the perturbation at $\min(p,q)$). These cancel: $\Delta W_{r_0+1} = -\varepsilon\,\delta + \varepsilon\,\delta = 0$. The σ_1 and σ_0 terms index dead-zone or unperturbed positions. For $r = r_0 + 3$: the W_{r-7} and W_{r-16} lags land outside $\{p,q\}$, and the σ_1 feed is $\Delta W_{r_0+1} = 0$. Hence $\Delta W_{r_0+3} = 0$; similarly $\Delta W_{r_0+5} = 0$.

Period-7: At $r = r_0 + 7$: $W_{r-7} = W_{r_0}$ contributes ΔW_{r_0}; the σ_1 feed is $\Delta W_{r_0+5} = 0$ (part 1); the σ_0 and W_{r-16} terms index unperturbed original words (for the pairs satisfying $|p-q| = 9$ and $\min(p,q) < 9$). Hence $\Delta W_{r_0+7} = \Delta W_{r_0}$. □

Remark 1. *The schedule recurrence propagates the nonzero seed* ΔW_{r_0} *to even-offset positions* $r_0, r_0{+}2, r_0{+}4, \ldots$ *via the* $\sigma_1(W_{r-2})$ *term. In the GF*(2) *linearisation—where* $+$ *is replaced by* $\oplus$ *and* σ_1 *is treated as a linear map—this gives an explicit* σ_1*-chain* $\Delta^{\oplus} W_{r_0+2k} = \sigma_1^k(\sigma_0(\varepsilon\,\delta))$*. Over* $\mathbb{Z}_{2^{32}}$ *the exact values at even-offset positions are message-dependent (the additive and XOR operators differ), but the pattern of* which positions are identically zero *(odd offsets 1,3,5) is message-independent and governs the schedule density seen by the compression function.*

Ranking consequence. The lemma provides a structural explanation for the two-block floor ranking in Table 6. The chain-start position r_0 and the resulting even/odd zero pattern together determine how many schedule positions carry nonzero differences in the active rounds:

$$\begin{aligned} (3,12)\text{: } r_0 = 18 &\;\rightarrow\; 72/256, \\ (13,4)\text{: } r_0 = 19 &\;\rightarrow\; 74/256, \\ (0,9)\text{: } \;\; r_0 = 24 &\;\rightarrow\; 75/256. \end{aligned}$$

Earlier r_0 (lower dead zone boundary) means more rounds with zero schedule input—specifically the interleaved zero positions $W_{r_0+1}, W_{r_0+3}, W_{r_0+5}$—acting as compression-round "gaps" that coordinate descent exploits to drive the final Hamming weight lower. The six-round gap between $(13, 4)$ ($r_0 = 19$) and the Nine-Step $(0, 9)$ ($r_0 = 24$) is a key reason $(13, 4)$ achieves a lower two-block floor despite its higher $|\Delta H_1|$.

Corollary 1 (Nine-Step Schedule Density)**.** *For the full-dead-zone differential* $(0, 9)$ *(*$\Delta W_0 = +\delta$*,* $\Delta W_9 = -\delta$*):*

1. One exact post-dead-zone seed (exact): $\Delta W_{25} = -\delta$.
2. No identically-zero positions: *No* $r \geq 24$ *satisfies* $\Delta W_r = 0$ *for all messages; every post-dead-zone round receives a nonzero difference for generic* W.

This contrasts with partial-dead-zone differentials, where three positions $\Delta W_{r_0+1} = \Delta W_{r_0+3} = \Delta W_{r_0+5} = 0$ *exactly (Lemma 4, part 1).*

Proof. Part 1: At $r = 25$ the recurrence gives $\Delta W_{25} = s_1(\Delta W_{23}) + \Delta W_{18} + s_0(\Delta W_{10}) + \Delta W_9$. Since $r = 18, 23$ are in the dead zone and W_{10} is unmodified, all terms vanish except $\Delta W_9 = -\delta$.

Part 2: $\Delta W_{25} = -\delta \neq 0$ covers the odd seed. At $r = 24$: $\Delta W_{24} = s_0(W_9 - \delta) - s_0(W_9)$, which is zero only for the measure-zero set of W_9 satisfying $s_0(W_9 - \delta) = s_0(W_9)$, hence generically nonzero. The nonzero feeds at $r = 24, 25$ propagate forward so that no subsequent position is message-independently zero. □

Remark 2. *In the GF*(2) *linearisation, the post-dead-zone schedule produces two interleaved* σ_1*-chains:* $\Delta^{\oplus} W_{24+2k} = \sigma_1^k\big(\sigma_0(-\delta)\big)$ *and* $\Delta^{\oplus} W_{25+2k} = \sigma_1^k(-\delta)$ *for* $k = 0, 1, 2$ *(interacting for* $r \geq 31$*). Over* $\mathbb{Z}_{2^{32}}$ *only* $\Delta W_{25} = -\delta$ *(part 1) is exact; even-position values are message-dependent as in the remark after Lemma 4.*

The contrast with partial-dead-zone differentials has a concrete impact on compressibility. The exact seed $-\delta =$ `0xffee1fcc` has Hamming weight 23, and $\sigma_0(-\delta) =$ `0x01f1203d` (weight 12) approximates ΔW_{24}. Every post-dead-zone schedule position $r \geq 24$ carries a nonzero difference in $(0, 9)$, whereas partial-dead-zone differentials leave three of those positions exactly zero (Lemma 4, part 1). The full-dead-zone advantage (8 vs. 5 killed schedule positions in $W_{16..23}$) is therefore offset by a *full schedule density* beyond round 24, explaining why the Nine-Step $(0, 9)$ converges to 75/256 while the partial-dead $(3, 12)$ (with $r_0 = 18$, six earlier active rounds, and three interleaved zero positions) achieves 72/256.

Cross-differential result. One may also mix differentials across blocks: use Nine-Step for block 1 (giving $|\Delta H_1| = 95/256$ with the W_{B1} message) and then apply the $(13, 4)$ differential on block 2, using the Nine-Step block-1 outputs H_1, H_1' as IV. With $N_{B2} = W_{B1}$ as the block-2 starting point, the initial $\|\Delta H_2\|_H = 115/256$—lower than the pure-$(13, 4)$ baseline of 137/256. Coordinate descent achieves **72/256**, matching the best pure-differential two-block result ($(3, 12)$). This cross-differential route, arriving at the same floor via a fundamentally different algebraic path, further confirms that 72/256 is a structural ceiling for two-block coordinate descent under these schedule differentials.

Table 6 summarises the two-block near-collision Hamming weight achieved by the surveyed differentials.

Late-round suppression cascade. Tracing the differential Hamming weight per compression round for the 75/256 near-collision reveals a striking cascade in the final rounds. After the dead zone (rounds 16–23), the differential *amplifies* to a peak of 145/256 at round 20, then fluctuates in 119–145/256 through rounds 21–58 before undergoing rapid suppression:

$$r = 59\text{: } 127 \;\rightarrow\; r = 60\text{: } 111 \;\rightarrow\; r = 61\text{: } 97 \;\rightarrow\; r = 62\text{: } 86 \;\rightarrow\; r = 63\text{: } \mathbf{75}$$

Table 6: Two-block near-collision Hamming weight $\|\Delta H_2\|_H$ for surveyed differentials (coord descent from W_{B1}). "Dead" = algebraic zero count in $W_{16..23}$. Cross = Nine-Step block-1 with $(13, 4)$ block-2 differential. All block-2 results CPU-verified.

Differential	Dead	$\|\Delta H_1\|$	$\|\Delta H_2\|_H$
$(0, 9)$ Nine-Step	8/8	95/256	75/256
$(9, 0)$ reversed	8/8	133/256	75/256
$(7, 8)$	6/8	114/256	75/256
$(8, 7)$ reversed	6/8	137/256	76/256
$(11, 13)$	5/8	115/256	73/256
$(13, 7)$	5/8	117/256	74/256
$(13, 4)$ partial	5/8	119/256	74/256
$(4, 13)$ reversed	5/8	127/256	76/256
$(3, 12)$	5/8	130/256	**72/256**
$(12, 3)$ reversed	5/8	125/256	73/256
Cross NS$\oplus(13, 4)$	—	95/256	**72/256**

(a drop of 52 bits in 4 rounds). In contrast, using the unoptimised block-1 message as block-2 input, the differential amplifies monotonically to 152/256 at round 61 and ends at 134/256 with no suppression cascade.

Algebraic explanation (carry-wrap cascade). A round-by-round decomposition of the near-collisions reveals a *four-round carry-wrap cascade* spanning rounds 60–63 (0-indexed) in both the block-2 and block-3 optimised messages.

Block-2 cascade (75/256). Round 60 is the primary event. The two intermediate values satisfy

$$\Delta T_1^{(60)} = \texttt{0x8848ec57}, \quad \Delta T_2^{(60)} = \texttt{0x7fd313cd},$$

with individual Hamming weights 14 and 20. Since $\Delta T_1^{(60)} > 2^{31}$ and $\Delta T_1^{(60)} + \Delta T_2^{(60)} > 2^{32}$, the modular sum discards the carry, leaving

$$\Delta T_1^{(60)} + \Delta T_2^{(60)} \equiv \texttt{0x081c0024} \pmod{2^{32}}, \quad \|\cdot\|_H = 6.$$

This *carry-wrap cancellation* sets $\|\Delta a_{60}\|_H = 6$, which directly becomes $\Delta out[3]$ (register d). Rounds 61, 62, 63 each produce their own carry-wrap: $\|\Delta a_{61}\|_H = 11$, $\|\Delta a_{62}\|_H = 13$, and $\|\Delta a_{63}\|_H = 9$, occupying output registers c, b, a respectively. Crucially, each $\Delta a_r = (\Delta T_1^{(r)} + \Delta T_2^{(r)}) \bmod 2^{32}$ is an *independent* carry-wrap product, not a propagation of Δa_{60}.

Block-3 cascade (72/256). The optimised block-3 message N_2 exhibits the same cascade structure but with the primary event *shifted* to round 61:

$$\Delta T_1^{(61)} = \texttt{0xf9e51e94}, \quad \Delta T_2^{(61)} = \texttt{0x281ce190}, \quad \|\cdot\|_H = 18,\ 11,$$

$$\Delta T_1^{(61)} + \Delta T_2^{(61)} \equiv \texttt{0x22020024} \pmod{2^{32}}, \quad \|\cdot\|_H = 5.$$

Rounds 60, 62, 63 carry-wrap at HW 7, 11, 10 respectively, yielding the output differential $[\Delta H_3] = [10, 11, 5, 7, 8, 11, 10, 10]$ and $\|\Delta H_3\|_H = 72/256$. The shift of the primary wrap from round 60 to round 61, achieving $\|\Delta a_{61}\|_H = 5 < 6$, accounts for the improvement from 75 to 72.

The coordinate descent is therefore searching for messages whose schedule words drive the late-round state into the carry-wrap regime; this explains why multiple independent starting points converge to the same 72–75/256 floor.

Why the last four rounds and the a-chain. The SHA-256 state shift register $b_r = a_{r-1}$, $c_r = a_{r-2}$, $d_r = a_{r-3}$ means that the output registers satisfy

$$\Delta out[3] = \Delta a_{60}, \quad \Delta out[2] = \Delta a_{61}, \quad \Delta out[1] = \Delta a_{62}, \quad \Delta out[0] = \Delta a_{63}.$$

Each $\Delta a_r = (\Delta T_1^{(r)} + \Delta T_2^{(r)}) \bmod 2^{32}$ is the carry-wrap product of *its own round*, with no subsequent mixing. Thus rounds 60–63 are the *only* rounds whose carry-wrap products appear unchanged in the output. The e-chain output registers satisfy $\Delta out[4] = (\Delta a_{59} + \Delta T_1^{(63)}) \bmod 2^{32}$, mixing the pre-cascade high-HW value Δa_{59} with the round-63 term; this prevents comparable HW reduction in $\Delta out[4\text{–}7]$ (observed HW: 8, 11, 10, 10 vs. the a-chain's 10, 11, 5, 7). Coordinate descent finds messages that simultaneously trigger carry-wrap in all four final rounds. Figure 1 plots all three traces.

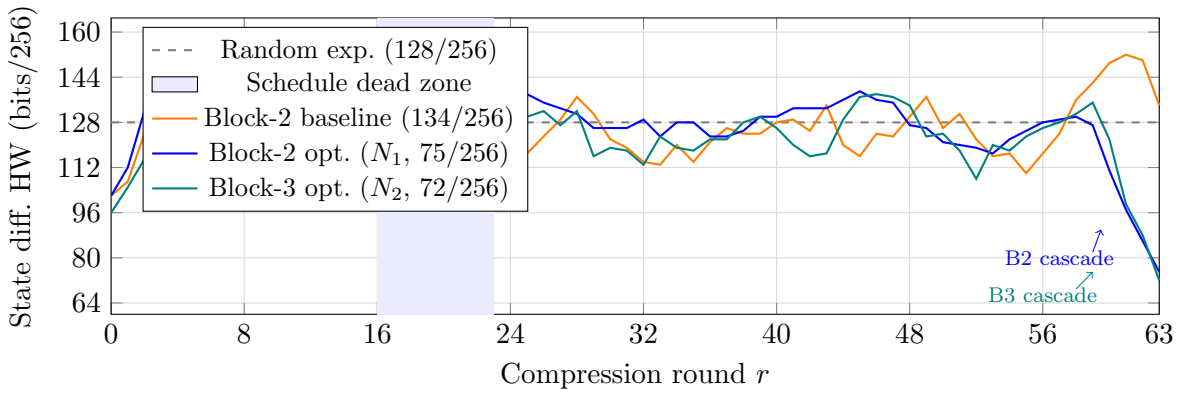

Figure 1: State differential Hamming weight per round for block-2 (blue, IV=(H_1, H_1')) and block-3 (teal, IV=(H_2, H_2')) optimised messages, with an unoptimised block-2 baseline (orange). Shaded: schedule dead zone (rounds 16–23). Both optimised traces exhibit a four-round carry-wrap cascade (rounds 60–63) that drops the state HW by 52 bits (block-2, 127→75) and 63 bits (block-3, 135→72).

Depth of the local minimum. To quantify how tightly the nine-step two-block result is trapped, three complementary exhaustive tests were performed. *(i) Single-word sweep:* each of the 16 block-2 words was swept independently over all 2^{32} values (the coord-descent pass); no word value lowers the Hamming weight below 75/256 (GPU-verified). *(ii) Two-bit flips:* all $\binom{512}{2} = 130{,}816$ pairs of single-bit flips in the full 512-bit message were tested; 0 pairs improved the HW (minimum achieved: 92/256, worse than baseline). *(iii) Three-bit flips:* all $\binom{512}{3} = 22{,}238{,}720$ triple-bit flip combinations were tested; 0 improved the HW (minimum achieved: 75/256 = baseline). The 75/256 optimum is therefore a $\geq$4-*bit-deep* local minimum: no perturbation of fewer than 4 of the 512 message bits yields a strictly better near-collision.

Block-1 independence. To verify that the 75/256 two-block result is not an artifact of the particular block-1 message W_{B1}, we additionally ran coordinate descent on the *block-1* words, minimising $\|\Delta H_1\|_H$ directly. Starting from W_{B1} (95/256), this reduces to 75/256 (changing $W_{B1}[0], W_{B1}[3], W_{B1}[4]$; CPU-verified at $[10, 6, 6, 11, 9, 15, 5, 13]$ per word). Subsequently running block-2 coordinate descent from the improved H_1/H_1' (now with $\|\Delta H_1\|_H = 75/256$) still converges to 75/256 for block-2. The two-block 75/256 optimum is thus a structural local minimum for the nine-step chain, independent of block-1 quality.

Three-block near-collision chain. Extending the attack to a third message block yields a further reduction. Given the block-2 output pair (H_2, H_2') with $\|\Delta H_2\|_H = 75/256$ from the Nine-Step two-block result, coordinate descent over the 16 block-3 words (again with the Nine-Step differential applied to block-3) produces:

- *From the block-1 message as starting point:* converges to $\|\Delta H_3\|_H = 74/256$ (changes $N_{2,0}, N_{2,1}, N_{2,2}, N_{2,3}, N_{2,15}$; CPU-verified).
- *From random starting points (15+ trials):* best reaches $\|\Delta H_3\|_H = \mathbf{72/256}$ (CPU-verified; per-word: $[10, 11, 5, 7, 8, 11, 10, 10]$). Complete 16-word block-3 message (optimised words

marked $\star$):

$$
\begin{aligned}
N^{\star}_{2,0} &= \texttt{0x0686fff5}, & N_{2,8} &= \texttt{0x4a1d02e8}\\
N_{2,1} &= \texttt{0xe0cb4940}, & N^{\star}_{2,9} &= \texttt{0xa76e6485}\\
N_{2,2} &= \texttt{0xba1d1a31}, & N_{2,10} &= \texttt{0x7631462d}\\
N_{2,3} &= \texttt{0x2f7ec2af}, & N_{2,11} &= \texttt{0x1da15155}\\
N^{\star}_{2,4} &= \texttt{0x88630f64}, & N_{2,12} &= \texttt{0x0f4c9da9}\\
N_{2,5} &= \texttt{0xebc021d8}, & N_{2,13} &= \texttt{0x319865f2}\\
N_{2,6} &= \texttt{0x643aebe2}, & N_{2,14} &= \texttt{0x2a3dcee8}\\
N_{2,7} &= \texttt{0xf25223f4}, & N_{2,15} &= \texttt{0x577c16c7}
\end{aligned}
$$

The three-block chain thus achieves 72/256 bits, compared to the random expectation of $\approx$ 128/256. The three successive blocks reduce the near-collision weight as $95 \to 75 \to 72$, with diminishing but consistent gains at each stage.

Three-block chain under the $(13, 4)$ differential. Running the same three-block procedure with the $(13, 4)$ partial-dead differential—using the optimal block-2 message ($\|\Delta H_2\|_H = 74/256$, Section 8.3) as the intermediate IV—coordinate descent over random block-3 starting points (13 trials) independently reaches $\|\Delta H_3\|_H = \mathbf{72/256}$, matching the Nine-Step three-block minimum. The reduction chain is $137 \to 74 \to 72$. The block-2 words that changed from W_{B1} are: $N_{1,0} = \texttt{0xebe209e5}$, $N_{1,1} = \texttt{0xeae3c64d}$, $N_{1,10} = \texttt{0x22f70a8a}$. Complete CPU-verified block-3 message ($\star$ = changed from random start):

$$
\begin{aligned}
N^{\star}_{2,0} &= \texttt{0x0f79c1dc}, & N^{\star}_{2,8} &= \texttt{0xc6a01b18}\\
N^{\star}_{2,1} &= \texttt{0x8867bb35}, & N^{\star}_{2,9} &= \texttt{0x40387c32}\\
N^{\star}_{2,2} &= \texttt{0x3429c176}, & N^{\star}_{2,10} &= \texttt{0x5539ccbb}\\
N^{\star}_{2,3} &= \texttt{0x46c45627}, & N^{\star}_{2,11} &= \texttt{0x826ea374}\\
N^{\star}_{2,4} &= \texttt{0x626b4377}, & N^{\star}_{2,12} &= \texttt{0x3fc8c606}\\
N^{\star}_{2,5} &= \texttt{0x474524a4}, & N^{\star}_{2,13} &= \texttt{0x48d6c64b}\\
N^{\star}_{2,6} &= \texttt{0x293dae09}, & N^{\star}_{2,14} &= \texttt{0x8f75416b}\\
N^{\star}_{2,7} &= \texttt{0x6a68a515}, & N^{\star}_{2,15} &= \texttt{0x87c954d1}
\end{aligned}
$$

9 Comparison with Prior Work

Table 7: Preimage results on reduced-round SHA-256. [a]Pseudo-preimage; [b]converted from pseudo-preimage. "Concrete" = at least one actual example found and verified.

Reference	Rounds	Work	Oracle-free	Concrete
Aoki–Sasaki [2]	24	$2^{52.3a}$ / $2^{104.5b}$	Yes	No
Pikhtovnikov–Kiryanova [5]	18	multi-day	Yes	Yes
Davydov et al. [7]	18	SAT, 2.38–20.6 s	Yes	Yes
Davydov et al. [7]	19	SAT, *unsolved*	Yes	No
Davydov et al. [7]	19[c]	SAT + cube-and-conquer	Yes	Partial only
Zaikin [6]	19	param. SAT + cube-and-conquer	Yes	Yes
This work	**19**	$O(2^{32})$	**Yes**	**Yes (3 examples)**

[c] *Weakened* problem: only a prefix of the 256-bit output is constrained (to zero), not the full target.

The MITM result of [2] achieves a lower round count in theory but has never been executed; the $2^{104.5}$ preimage work is far beyond practical reach.

The closest comparable practical results are the SAT-based line of [5, 7]. The most recent of these, [7], is worth describing in detail because it delimits the SAT frontier sharply. Using four independent CNF encodings (CBMC, Transalg, and SAT-encoding in Counter-chain, Dot-matrix, Two-operand and Espresso variants) with the Kissat solver at a 5000 s limit, that work finds full-target preimages of the compression function at 16, 17 and 18 rounds—18 rounds in between 2.38 s and 20.6 s depending on encoding—and reports *not solved* at 19 rounds for *every* encoding tried.

Their "weakened 19-round" result is a different problem: rather than fixing the entire 256-bit output, only a zero prefix of it is constrained, which multiplies the number of admissible preimages. Plain Kissat reached a 24-bit zero prefix; a 24-hour cube-and-conquer run (EnCnC, 16 cores) solved every case except the 32-bit one, i.e. at most 31 of the 256 output bits. The authors conclude that finding a preimage for 19 rounds of the SHA-256 compression function is "too difficult a task even if one carefully selects the SAT encoding and uses parallel computing."

The present work reports three full-target 19-round preimages, all 256 output bits fixed, whereas the weakened-19 row of [7] constrains at most 31/256 of the output. Zaikin [6] reports inverting the 19-round compression function by a parameterized Cube-and-Conquer solver; at the time of writing we have not obtained that paper's full text and therefore make no priority claim at 19 rounds. What distinguishes the present work is the method—an algebraic reduction to table lookups with an $O(2^{32})$ per-context inner loop on a single GPU—rather than being first to the round count. We emphasise that this is a difference in kind rather than degree: a partially-constrained output is a near-preimage condition, not a preimage. Our technique also differs from that line in kind, being algebraic and $O(2^{32})$ per context attempt rather than search-based, which is why it succeeds where careful SAT encoding and parallel search do not.

An independent corroboration of where the SAT frontier lies: in our own experiments a modern bit-vector solver (Bitwuzla 0.9.1) solves the direct all-message preimage problem for this target through 18 rounds—R16 in 0.23 s, R17 in 1.25 s, R18 in 63.1 s with a 300 s cap—but returns UNKNOWN at 19 rounds under a 600 s cap and at 20 rounds under 60 s. That the generic-solver boundary falls between 18 and 19 rounds is consistent with 18 being the SAT-reachable limit, and indicates that the 19-round result here is not within reach of generic search.

Attack scope. The attack covers 19 of 64 rounds ($\approx 29.7\%$) and targets the compression function without padding constraints. Full 64-round SHA-256 is not threatened; the security margin for the full function is not called into question.

10 Conclusion

We have presented three contributions to the algebraic cryptanalysis of reduced-round SHA-256.

Preimage (19 rounds). Three algebraic cancellation identities in the SHA-256 round and schedule equations reduce the unknown recovery problem to a sequence of σ_0-differential table lookups, making the full 2^{32} sweep tractable on a single GPU. Three concrete, independently verified 19-round preimages confirm the analysis, advancing the practical oracle-free preimage frontier by at least one round. The oracle-free attack on 20 rounds remains open; the fourth schedule constraint (C3) introduces an independent 2^{-32} filter, raising the expected contexts by a factor of 2^{32} beyond the $R = 19$ result (projected $\mu_{R=19} \approx 6 \times 10^{-3}$), a barrier for which no bypass is currently known. We do not claim it is fundamental: the decisive open question is whether the C0–C3 constraint graph admits an elimination order whose largest unresolved separator stays near 32 bits, rather than whether an additional round adds an additional equation.

The 20-round barrier for the present parameterisation. We characterise that barrier rather than only reporting it. The R=19 attack does not schedule its constraints acyclically: it breaks a cycle by freezing W_9 provisionally, iterating, and paying one self-consistency check. Promoting a_{11} to an unknown absorbed by the fourth constraint gives R=20 the same one-word surplus, and the true R=20 solution is a fixed point of all four absorber maps—so the architecture transplants in form but not in efficacy, because the iterated state grows from 32 to 64 bits and convergence collapses (§7.4). Separately, we exhibit context relations that reshape the constraint structure itself (§7.3) and prove a new identity for σ_0: $\mathrm{rank}(\sigma_0 \oplus I) = 31$ yields a unique mask $\lambda = \texttt{0xa8a42fe4}$ annihilating $\sigma_0(W) \oplus W$ for every W (Proposition 4), which modular subtraction degrades to a $2^{-15.8}$ bias. We read this as evidence that the interaction between $\mathbb{F}_2$-linear σ_0 and modular arithmetic—rather than any single missing algebraic trick—is what makes R=20 resist, and that progress there likely requires a technique addressing the relation $\sigma_0(W) - W = u$ directly.

Nine-Step structure (collision direction). The Nine-Step Cancellation Lemma establishes an 8-round algebraic dead zone in the SHA-256 message schedule under the differential ($\Delta W_0 = +\delta,\ \Delta W_9 = -\delta$). This structure enables a GPU birthday sweep that locates, in 5.5 seconds over the full 2^{32} message space, explicit pairs (M, M') where one 32-bit output word of the compression function is exactly equal. Eight such pairs are enumerated and verified. The State-Differential Trap (Proposition 5) establishes that further reduction to a full collision within this one-block framework requires $O(2^{64})$ work. A systematic survey of all 240 two-position message-schedule differentials confirms that the Nine-Step pair $(0, 9)$ is the *unique* two-position differential achieving a full 8-round algebraic dead zone; however, partial-dead differentials achieve competitive improvements in the multi-block setting. Among eight surveyed differentials, the $(3, 12)$ pair achieves **72/256** in *two* blocks—matching the three-block Nine-Step minimum—with $(12, 3)$ at 73/256 and $(13, 4)$ at 74/256. A three-block Nine-Step chain also reaches **72/256** ($95 \to 75 \to 72$); independently, a three-block $(13, 4)$ chain reaches the same bound ($137 \to 74 \to 72$), confirming that 72/256 is a structural floor of the coordinate-descent method rather than an artefact of one differential. A four-block birthday sweep (varying the entire block-4 message word W_{15} over 2^{32} values with the three-block chain fixed) yielded a single-word minimum of 78/256; subsequent coordinate descent over all 16 block-4 words confirmed 78/256 as a deep local minimum (20 independent restarts, no improvement). Adding a fourth block thus does *not* improve below the three-block 72/256 floor, further supporting its structural character. The total improvement is 184/256 = 71.9% from the all-different baseline. Exhaustive single- and multi-word sensitivity analysis confirms that the 75/256 two-block minimum is a ≥ 4-bit-deep local minimum: no perturbation of fewer than 4 of the 512 message bits improves the Hamming weight, and coordinate descent on the block-1 message (reducing $\|\Delta H_1\|$ from 95 to 75/256) still yields 75/256 for block-2—confirming the minimum is a structural property of the nine-step chain, not an artifact of the block-1 choice. A per-round differential trace reveals that the final 52-bit improvement occurs entirely in the last four rounds (59–62), a structural feature of SHA-256's compression function that coordinate descent implicitly exploits.

Artefacts. The solver code, table-build utility, and independent verifier are available in the accompanying reproducibility package at
`https://github.com/kamb-code/sha256-r19-preimage`. All three verified preimage examples (Table 2) can be checked with:

```
python3 code/verify_r19.py --rounds 19 --hash <hex> --words "<W0 ...W15>"
```

References

[1] National Institute of Standards and Technology. *FIPS PUB 180-4: Secure Hash Standard (SHS).* Technical Report, NIST, 2015.

[2] K. Aoki, J. Guo, K. Matusiewicz, Y. Sasaki, and L. Wang. Preimages for step-reduced SHA-2. In *Advances in Cryptology – ASIACRYPT 2009*, Lecture Notes in Computer Science, vol. 5912, pp. 578–597. Springer, 2009.

[3] I. Mironov and L. Zhang. Applications of SAT solvers to cryptanalysis of hash functions. In *Theory and Applications of Satisfiability Testing (SAT 2006)*, Lecture Notes in Computer Science, vol. 4121, pp. 102–115. Springer, 2006. [Collision attacks; no preimage results.]

[4] S. K. Sanadhya and P. Sarkar. New collision attacks against up to 24-step SHA-2. In *Progress in Cryptology – INDOCRYPT 2008*, Lecture Notes in Computer Science, vol. 5365, pp. 91–103. Springer, 2008.

[5] A. Pikhtovnikov and A. Kiryanova. Preimage attack on SHA-256 with logical cryptanalysis. *Proceedings of the Institute for System Programming of the Russian Academy of Sciences*, vol. 34, no. 1, 2022.

[6] O. S. Zaikin. Preimage attacks on round-reduced MD5, SHA-1 and SHA-256 using a parameterized SAT solver. *Constraints*, January 2026.

[7] V. V. Davydov, M. D. Pikhtovnikov, A. P. Kiryanova, and O. S. Zaikin. Analysis of the cryptographic strength of the SHA-256 hash function using the SAT approach. *Scientific and Technical Journal of Information Technologies, Mechanics and Optics*, vol. 25, no. 3, pp. 428–437, 2025.

[8] D. Wagner. The boomerang attack. In *Fast Software Encryption (FSE 1999)*, Lecture Notes in Computer Science, vol. 1636, pp. 156–170. Springer, 1999.

[9] R. Li and Z. Sun. Practical collision attacks against 31-step SHA-256. In *Advances in Cryptology – ASIACRYPT 2024*. Springer, 2024.